

Basic Time Series Analysis

Asst.Prof.Dr. Supakit Nootyaskool

ITKMITL

Training plan

4 พฤษภาคม 2568	การวิเคราะห์ Time Series สำหรับผู้เริ่มต้น			
	09:00 - 12:00 น.	Topic 1: การพยากรณ์และการคาดการณ์อากาศระดับท้องถิ่น	ผศ.ดร. สุภกิจ นุตสกุล คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระ จอมเกล้าเจ้าคุณทหาร ลาดกระบัง	Online ผ่าน Zoom
		การรวบรวมและเตรียมข้อมูลอนุกรมเวลา (อุณหภูมิ ปริมาณฝน ความชื้น)		
		การวิเคราะห์แนวโน้ม (Trend) และความเป็นฤดูกาล (Seasonality)		
		การสร้างโมเดลพยากรณ์อย่างง่าย		
		การแปลผลและการนำเสนอผลการพยากรณ์		
		Workshop : "ระบบพยากรณ์สภาพอากาศท้องถิ่นเพื่อเกษตรกร" ที่รวบรวมข้อมูลสภาพอากาศในพื้นที่และสร้างโมเดลพยากรณ์ระยะสั้น (7-14 วัน) พร้อมคำแนะนำสำหรับเกษตรกรในการวางแผนกิจกรรมทางการเกษตร เช่น การให้น้ำ การหว่านเมล็ด หรือการเก็บเกี่ยว		
	12:00 - 13:00 น.	พักรับประทานอาหารกลางวัน	ผศ.ดร. สุภกิจ นุตสกุล คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระ จอมเกล้าเจ้าคุณทหาร ลาดกระบัง	
	13:00 - 16:00 น.	Topic 2: การติดตามสุขภาพและการวิเคราะห์แนวโน้ม		
		การเก็บรวบรวมข้อมูลสุขภาพอย่างต่อเนื่อง (น้ำหนัก ความดัน ระดับน้ำตาล อัตราการเต้นของหัวใจ)		
		การวิเคราะห์แนวโน้มและความสัมพันธ์ระหว่างปัจจัยสุขภาพต่างๆ		
		การตรวจจับความผิดปกติและการแจ้งเตือน		
		การสร้างรายงานและแดชบอร์ดเพื่อติดตามสุขภาพ		
		Workshop : "ระบบติดตามสุขภาพผู้สูงอายุในชุมชน" ที่รวบรวมข้อมูลสัญญาณชีพและกิจกรรมประจำวันของผู้สูงอายุ วิเคราะห์แนวโน้มเพื่อตรวจจับความเสี่ยงด้านสุขภาพล่วงหน้า และสร้างระบบแจ้งเตือนสำหรับผู้ดูแล		

Morning Section

การพยากรณ์และการคาดการณ์อากาศระดับท้องถิ่น

Topics

1. Understanding the time series analysis

- การรวบรวมและเตรียมข้อมูลอนุกรมเวลา (อุณหภูมิ ปริมาณฝน ความชื้น)
- การวิเคราะห์แนวโน้ม (Trend) และความเป็นฤดูกาล (Seasonality)

2. Time-series data form

3. Smoothing method

4. Decomposing method with Python

- การสร้างโมเดลพยากรณ์อย่างง่าย
- การแปลผลและการนำเสนอผลการพยากรณ์
- **Workshop** : “ระบบพยากรณ์สภาพอากาศท้องถิ่นเพื่อ การเกษตร” ที่รวบรวมข้อมูลสภาพอากาศในพื้นที่และสร้าง โมเดลพยากรณ์ระยะสั้น (7-14 วัน) พร้อมคำแนะนำสำหรับ เกษตรกรในการวางแผนกิจกรรมทางการเกษตร เช่น การให้น้ำการหว่านเมล็ด หรือการเก็บเกี่ยว

Resource

- https://www.ecmwf.int/en/forecasts/datasets?utm_source=chatgpt.com
- Visualizing and example
- <https://www.geeksforgeeks.org/analyzing-weather-data-in-r/>



1. Understanding Time Series Analysis



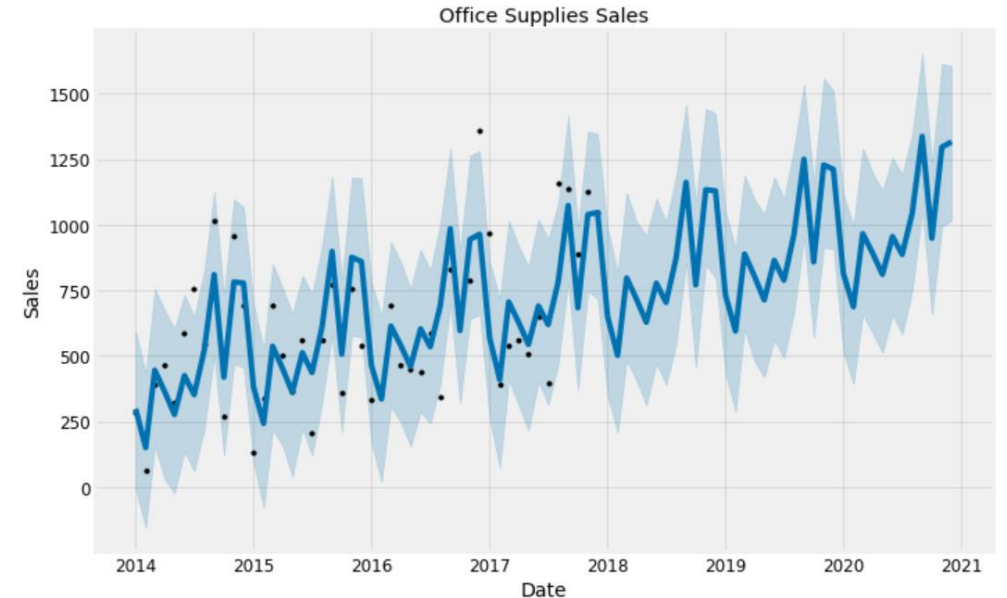
Time series data sources



Jasmin Shop



Month	Recreational goods
Jan 2000	134.1
Feb 2000	133
Mar 2000	136.2
Apr 2000	120.4
May 2000	126.2
Jun 2000	119.8
Jul 2000	122.5
Aug 2000	122.3
Sep 2000	131.2
Oct 2000	138.7
Nov 2000	158.4
Dec 2000	232.2
Jan 2001	143.1
Feb 2001	142.4
Mar 2001	152.6
Apr 2001	139.9
May 2001	135.6
Jun 2001	129.4
Jul 2001	138.5
Aug 2001	130.2



The time-series data sources presents are:

- Line plot
- The moving average is used to cut through noise and make smooth signal.

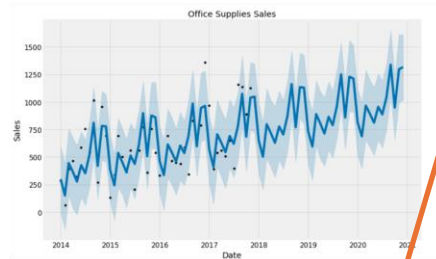
Time-series data related to a specific time duration.



Jasmin Shop



Month	Recreational goods
Jan 2000	134.1
Feb 2000	133
Mar 2000	135.2
Apr 2000	130.4
May 2000	126.2
Jun 2000	133.8
Jul 2000	122.5
Aug 2000	131.2
Sep 2000	136.7
Oct 2000	138.4
Nov 2000	232.2
Dec 2000	143.1
Jan 2001	142.4
Feb 2001	142.4
Mar 2001	137.6
Apr 2001	139.9
May 2001	135.6
Jun 2001	129.4
Jul 2001	138.5
Aug 2001	130.2



1999, 2000, ... , 2021 : Yearly

Q1, Q2, ... : Quarter

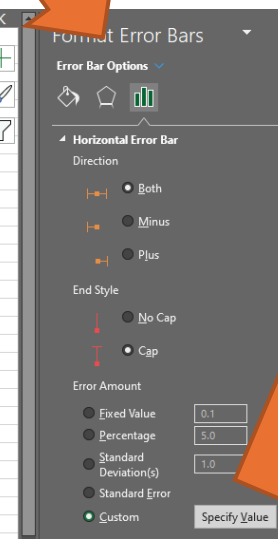
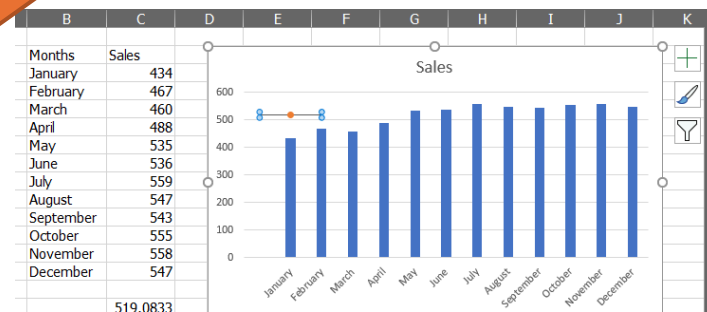
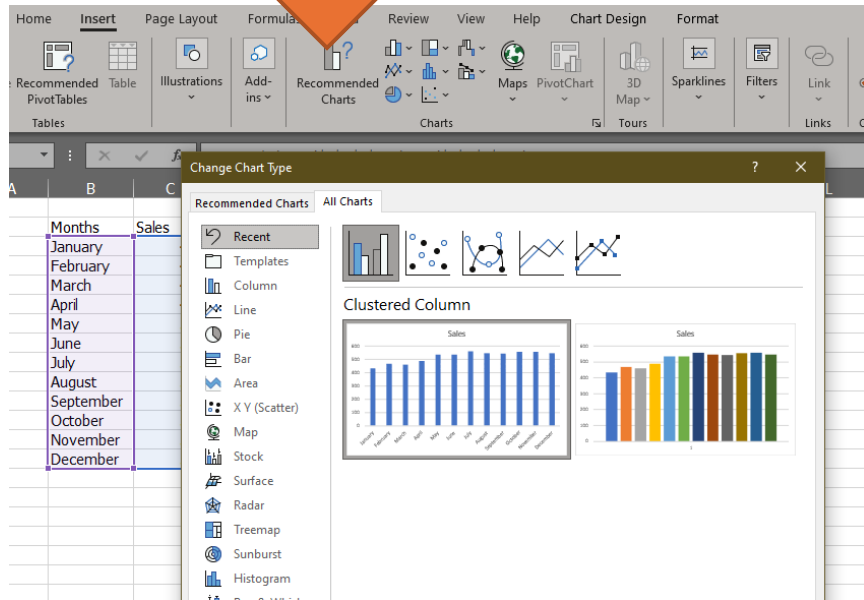
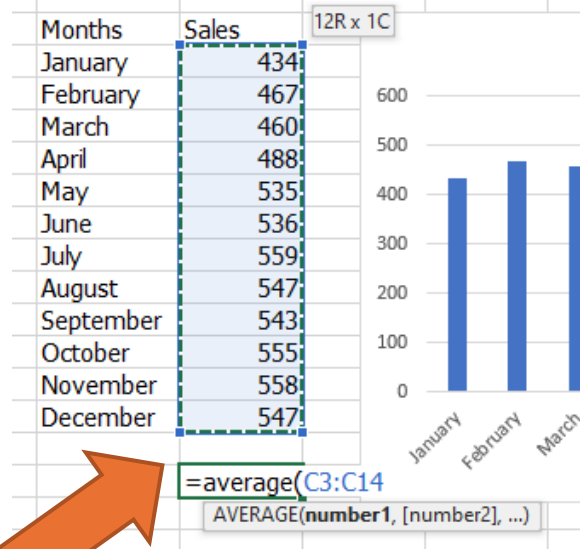
Jan, Feb, .., Dec : Monthly

W1, W2, .., W52 : Weekly

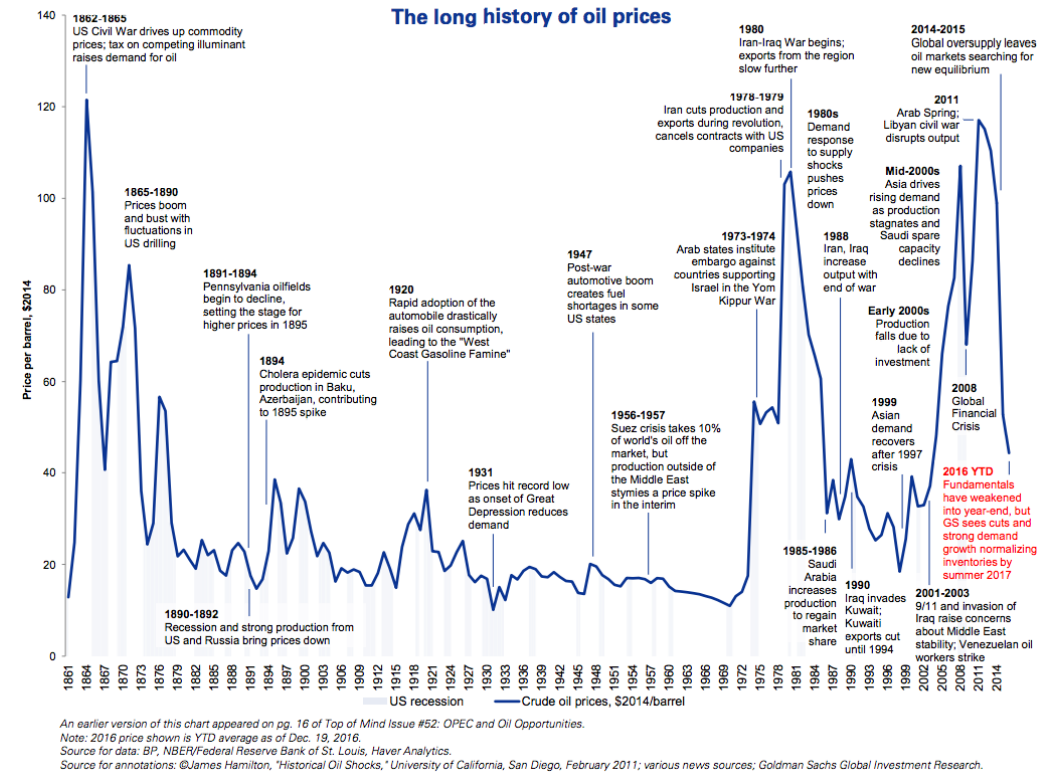
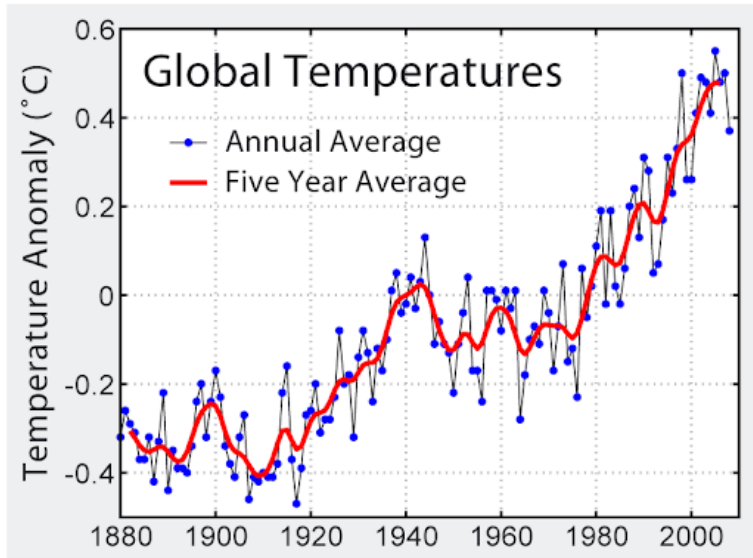
1 Oct – 30 Sep : Fiscal year

...

Activity: Plotting in Excel



Example Time-Series Data



Bitcoin Price at New Record High

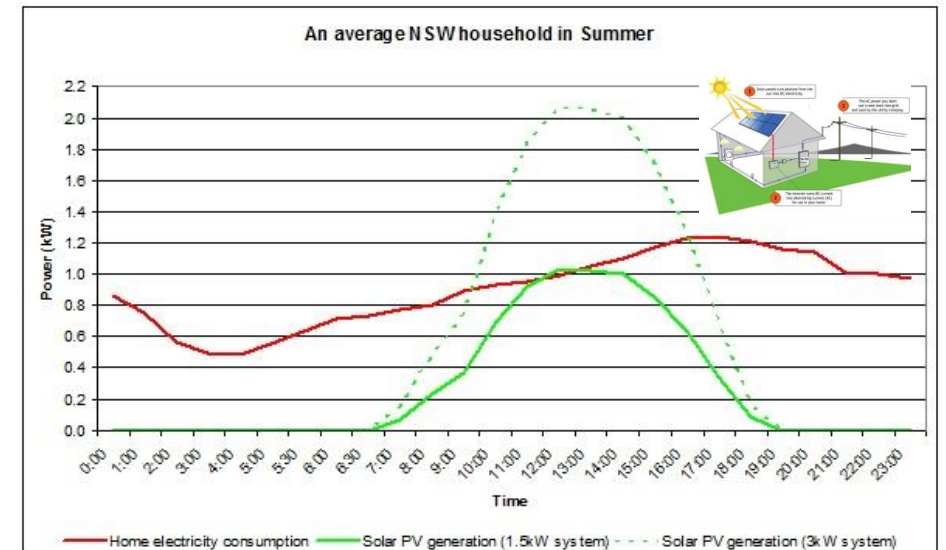
Daily Bitcoin price in U.S. dollars (2017-2021)*



* Closing price (latest data in range, UTC time)
Source: Coin Market Cap

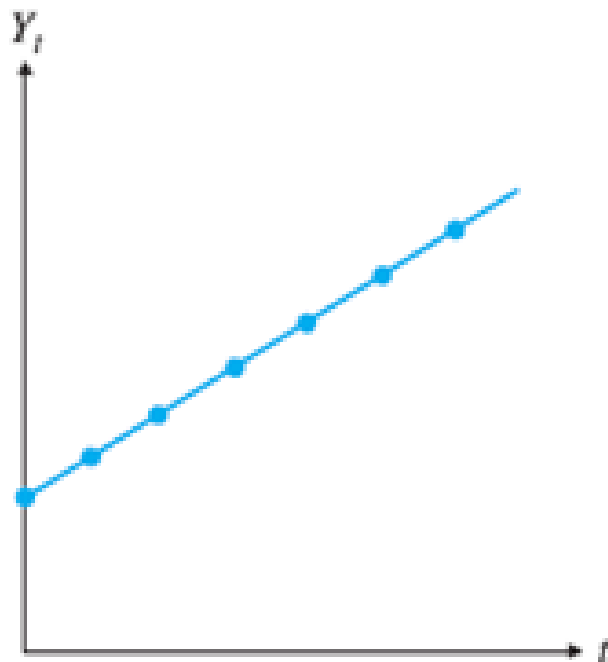


statista

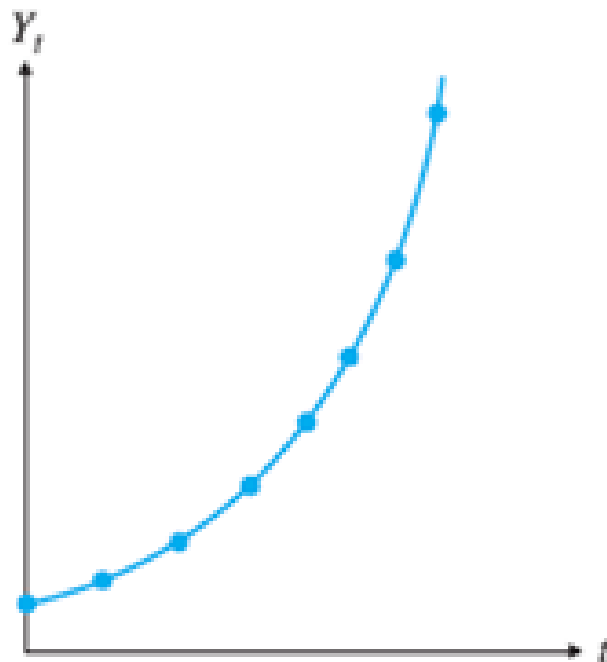


Time-series data components

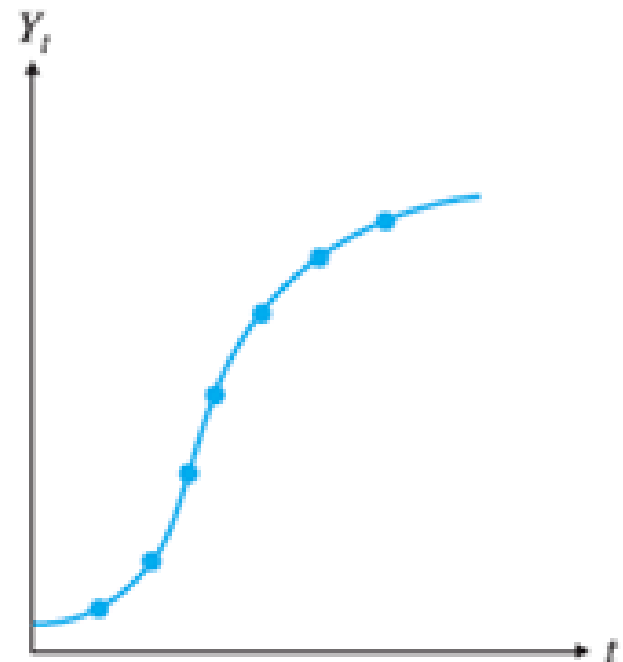
1. **Trend-line** แนวโน้ม It indicates the movement direction of the data-series, which can be moving up, down, or remaining unchanged.



(a) Linear trend



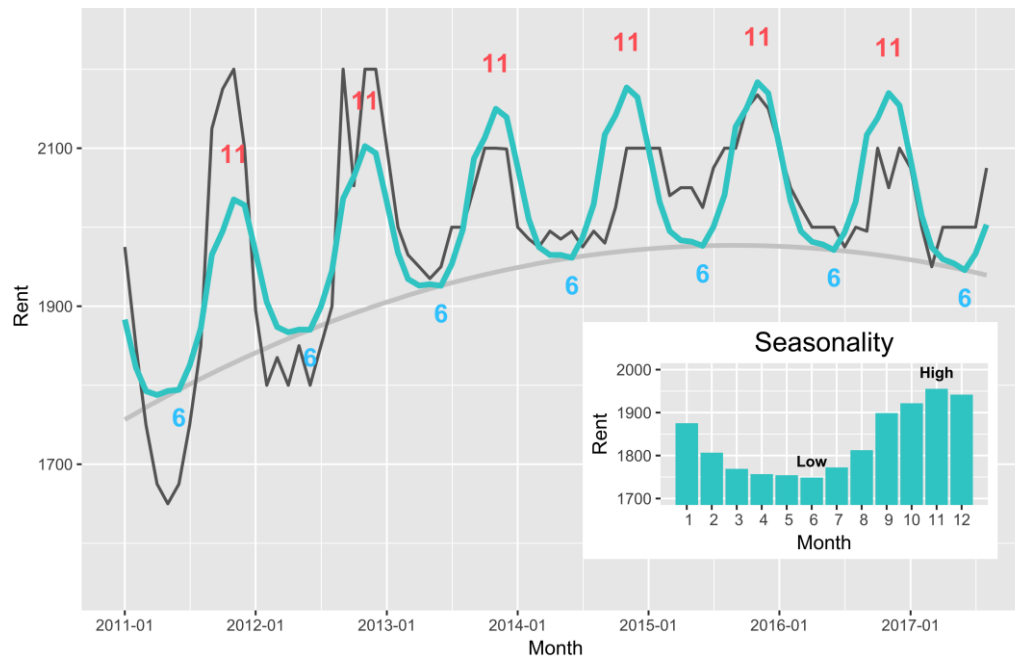
(b) Exponential trend



(c) S-shaped trend

Time-series data components

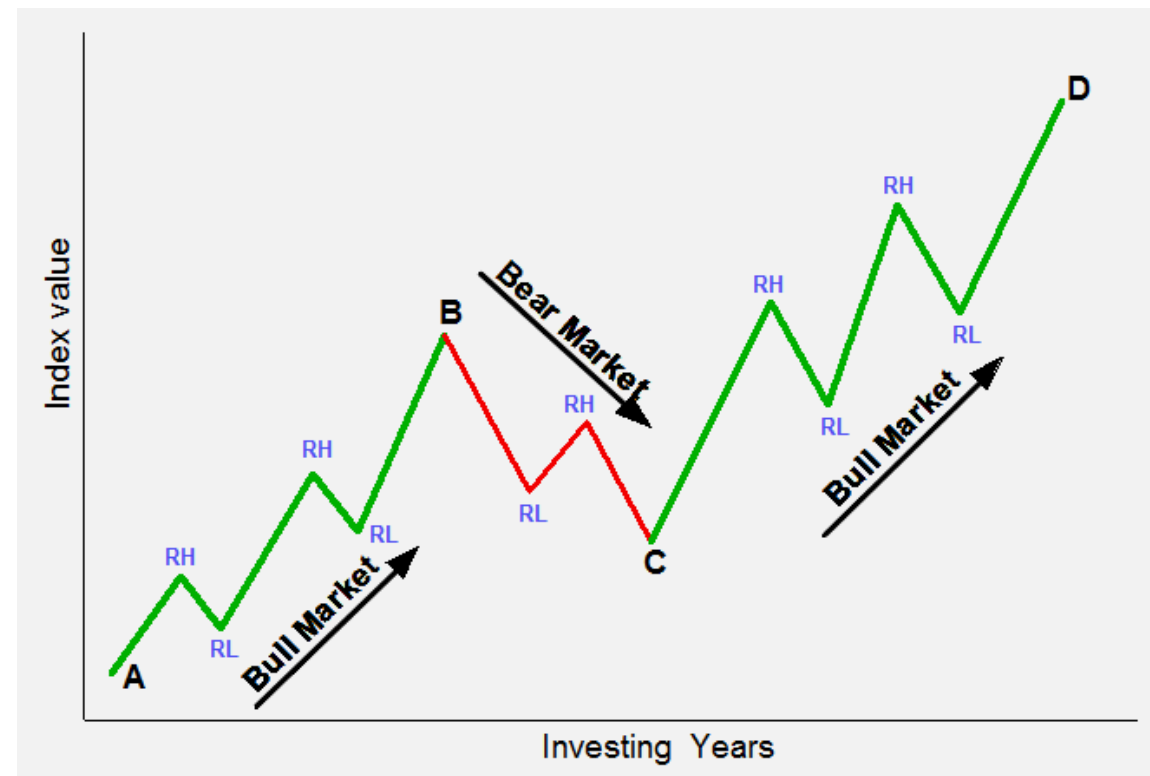
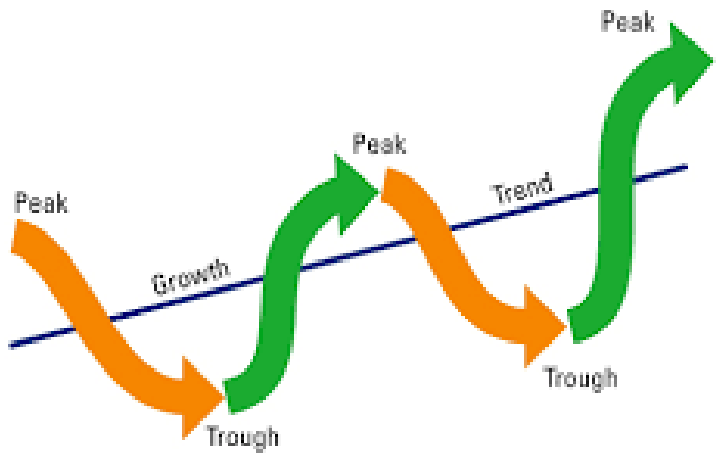
2. Seasonality ตามฤดูกาล It indicates the recurring cycle of up and down, which occurs at same time every year.



Time-series data components

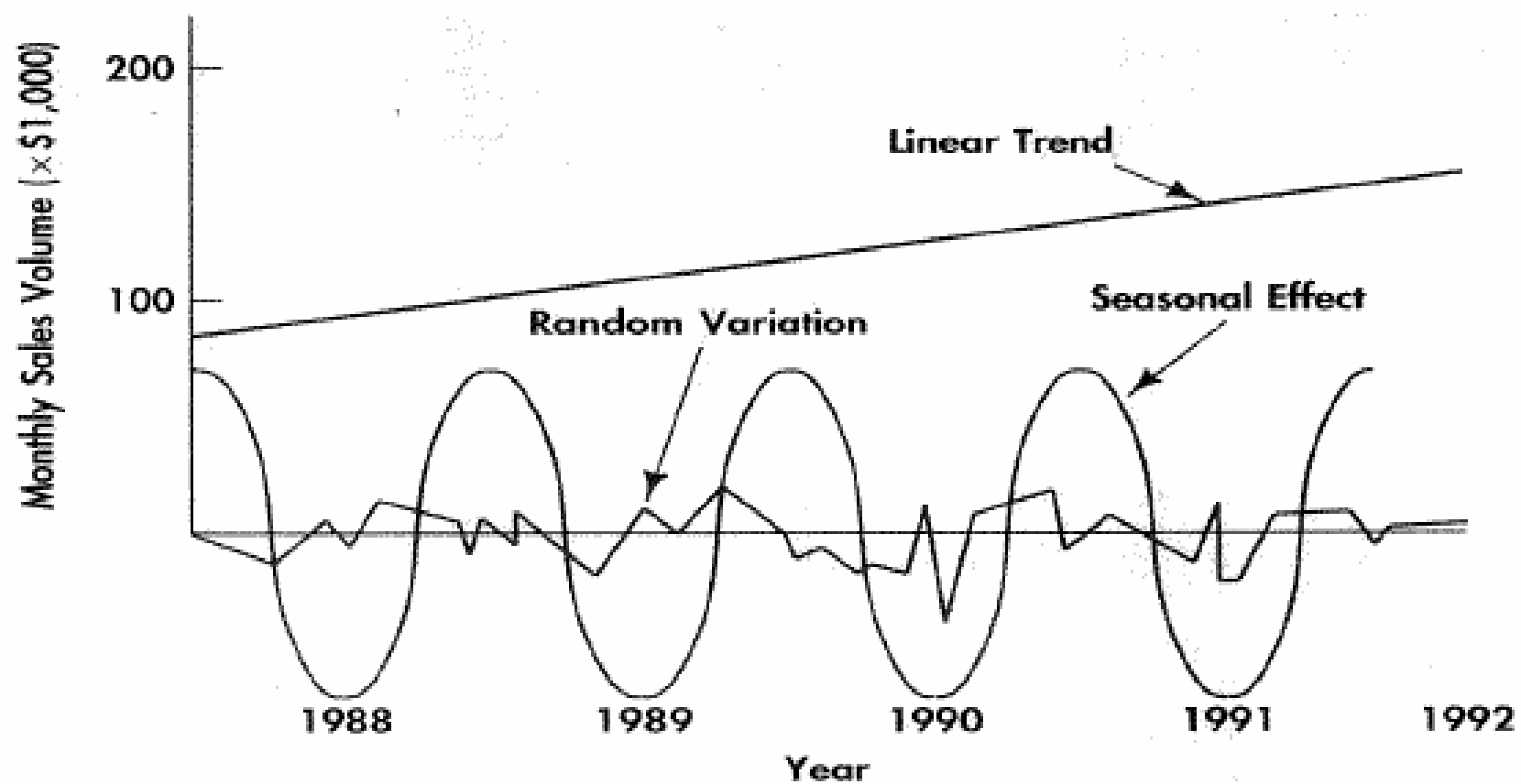


3. Cyclic ตามรอบ It indicates the recurring cycle of up and down that does not have the same length and does not occur at the same time, unlike seasonal trends.



Time series components

4. Random variation is the changing up or down that does not predicted the occurring.



Time series components

5. Irregularities is occurrence of unforeseen events: War, earthquake, flood, virus or illness.





2. Time-Series Data Equation Form



Time-series data is typically presented by two equation forms

1) Addition Equation

$$Y = T + S + C + I$$

Trend: T

Season: S

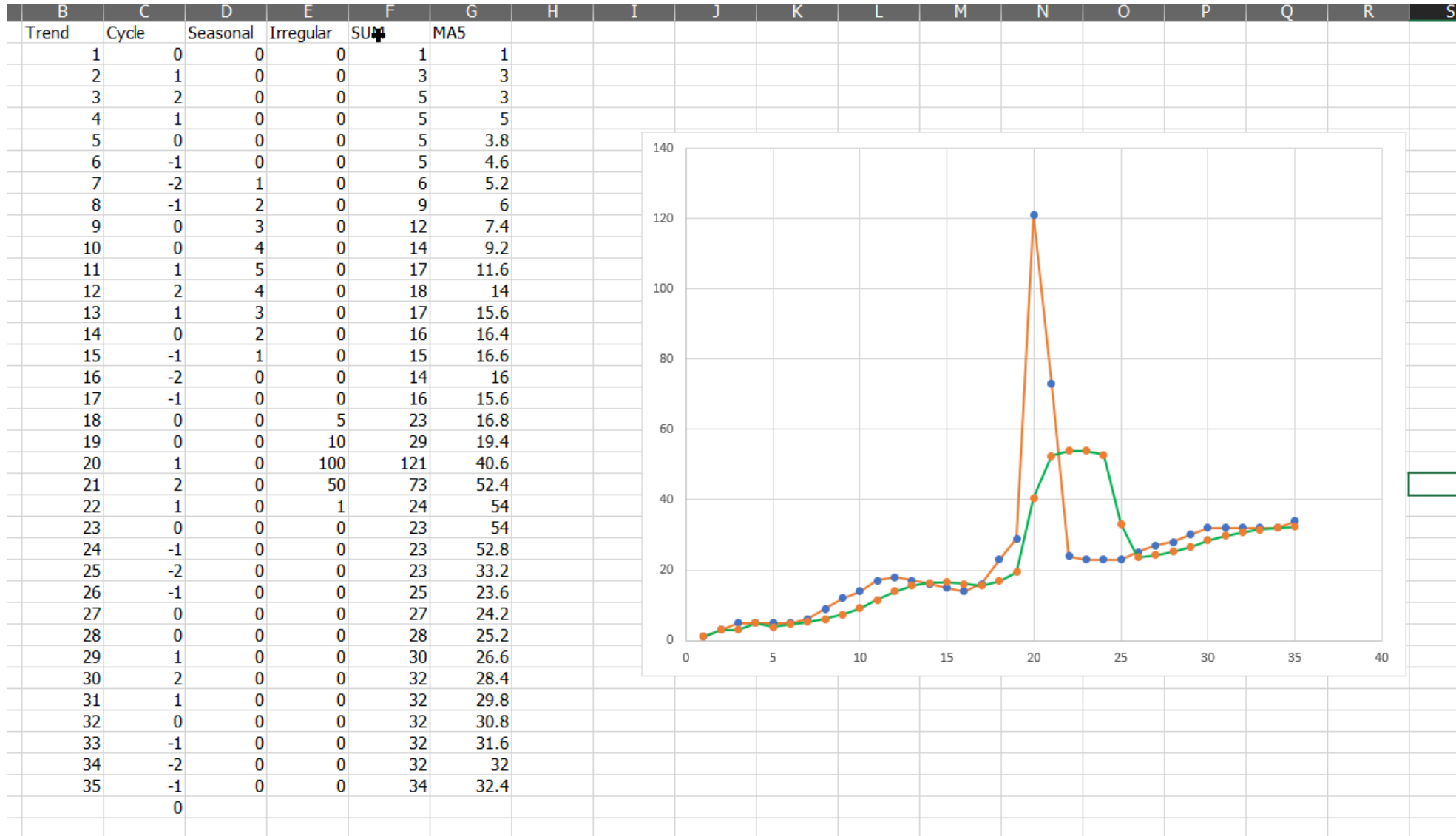
Cycle: C

Irregular: I

2) Multiplication Equation

$$Y = T \times S \times C \times I$$

Activity 1: Build the time-series data with Excel



Activity 2: Build the time-series data with Python

```
#Create a trend line
t = []
for i in range(0,100):
    t.append(i)
print(t)

#Create cyclic
import math
c = []
for i in range(0,100):
    c.append(math.sin(i/10)*2)
print(c)

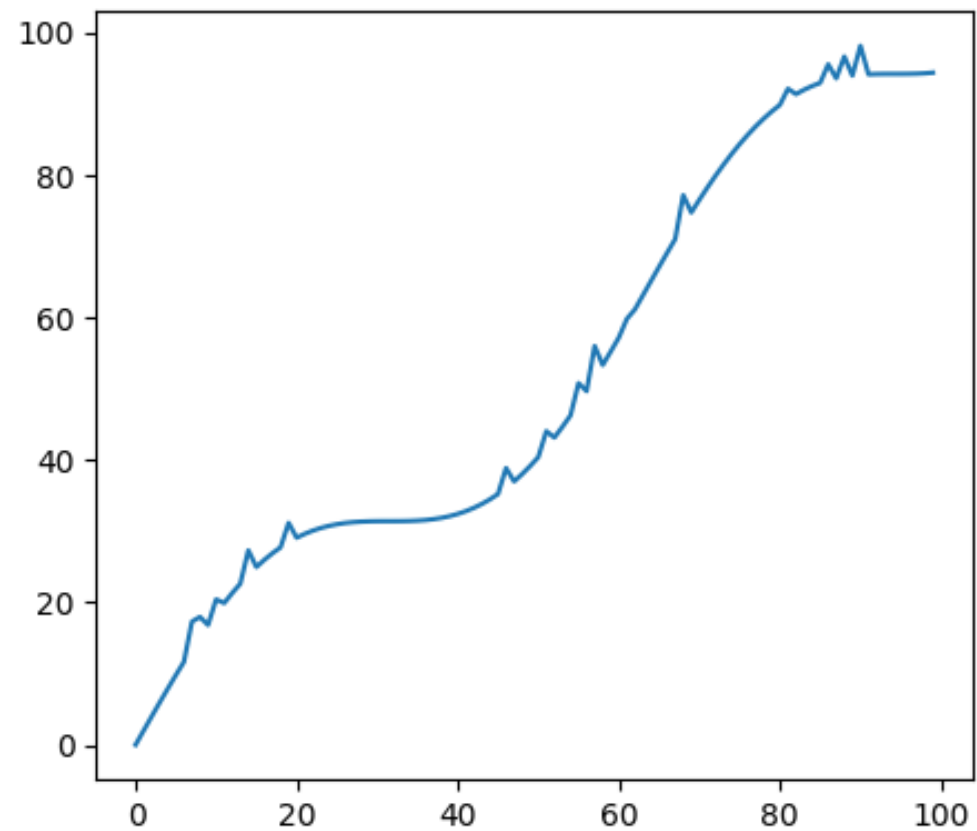
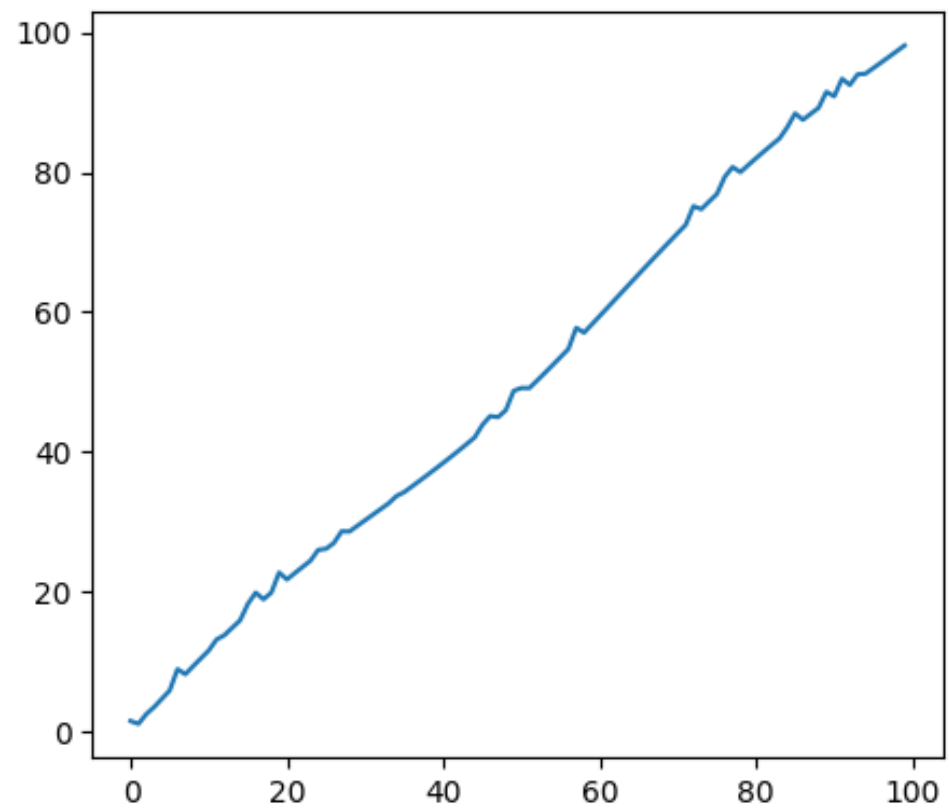
#Create random
import random as ran
r = []
for i in range(0,100):
    if (ran.random() > 0.8):
        r.append(ran.random()*2)
    else:
        r.append(0)
print(r)
```

```
sig = []
for i in range(0,100):
    sig.append(t[i]+c[i]+r[i])

print(sig)

#Plot
import matplotlib.pyplot as mpl
mpl.clf() #Clear plot
mpl.plot(range(0,100),sig)
mpl.show()

#Export to csv
import csv
with open('output.csv', 'w', newline = '\n') as
csvfile:
    my_writer = csv.writer(csvfile, delimiter = ",")
    my_writer.writerow(sig)
```





3. Smoothing Methods



Smoothing method

Smoothing is helpful in revealing patterns and trends in time-series data for prediction purposes.

This process involves "smoothening" out irregular fluctuations to uncover a more distinct pattern.

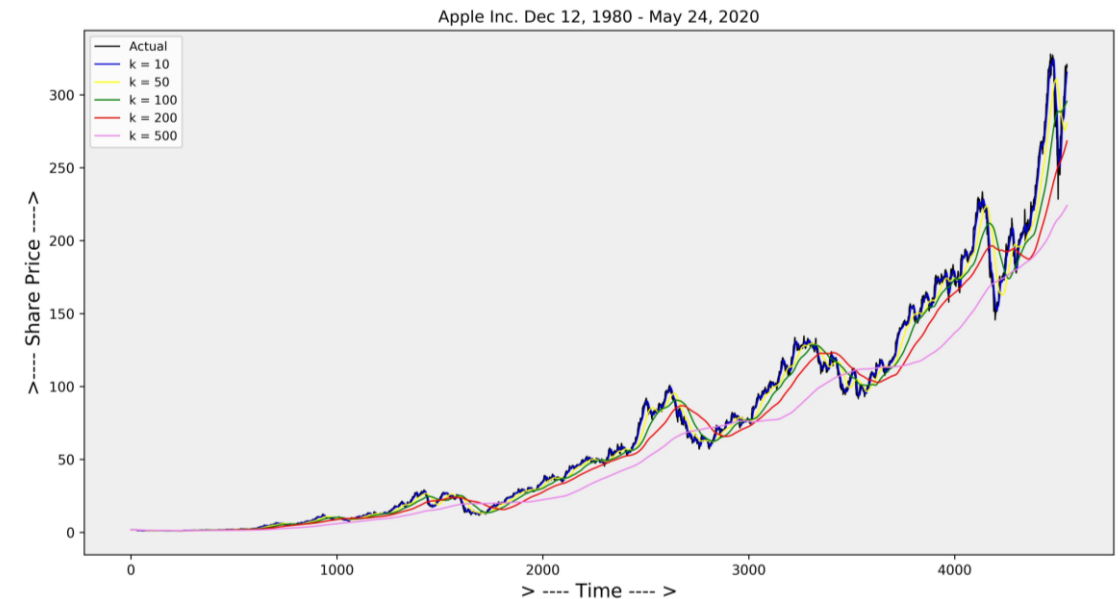
The methods used for smoothing include:

- Moving Averages
- Exponential Smoothing
- Least Squares Method

Moving average

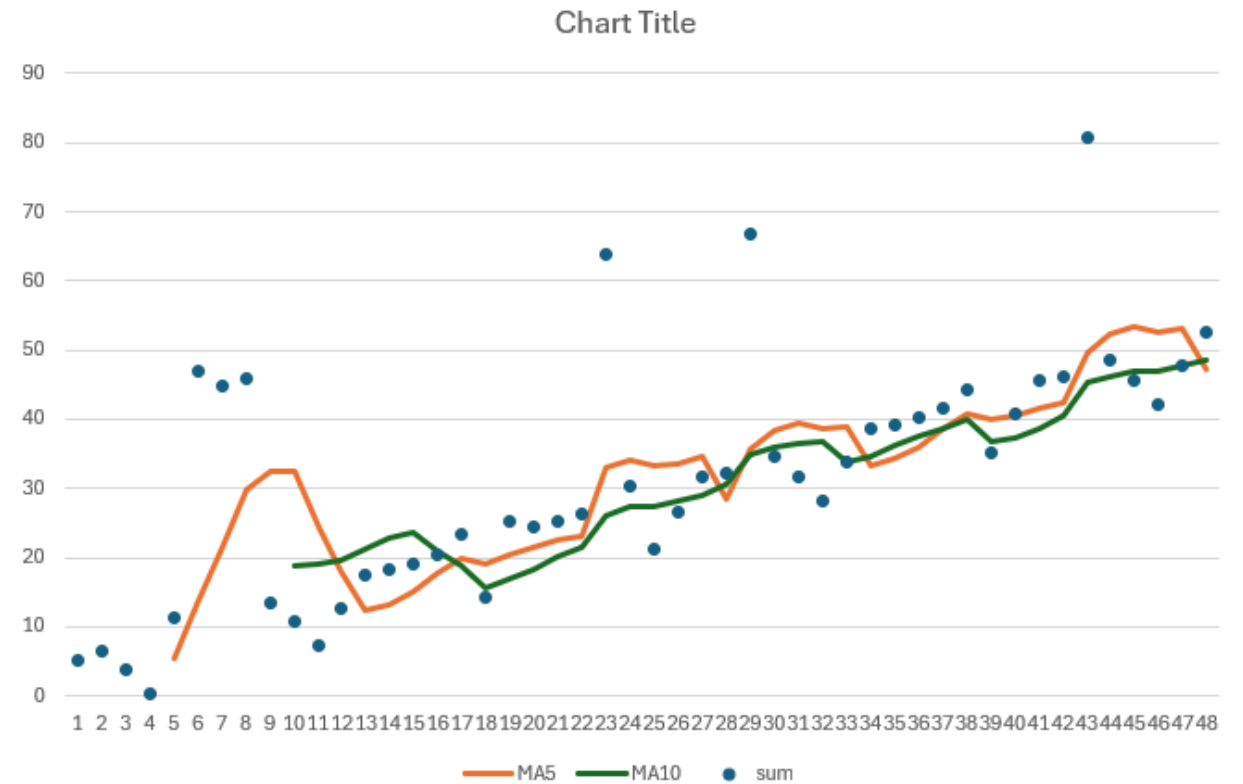
- The moving average method involves calculating the average value of data points at each point in time, creating a new dataset that represents the smoothed signal.

$$S_t = \frac{(X_{t-k} + X_{t-k+1} + X_{t-k+2} + \dots + X_t)}{k}$$



Activity 3: Moving average with Excel

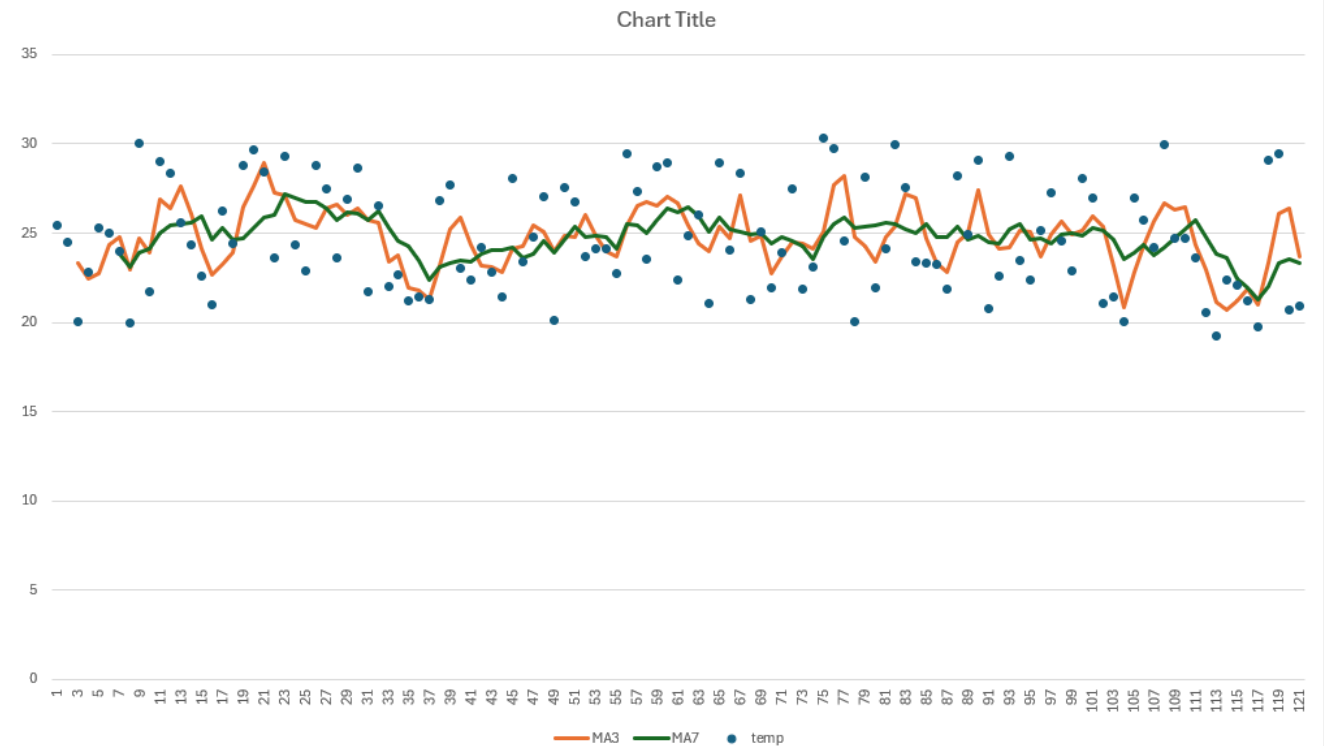
Trend	cycle1	cycle2	iragular	sum	MA5	MA10
1	1	4.207355	0	5.207355		
2	2	4.546487	36.3719	42.91838		
3	3	0.7056	0	3.7056		
4	4	-3.78401	0	0.215988		
5	3	0.7056	5.6448	11.3504	12.67955	
6	2	4.546487	0	10.54649	13.74737	
7	1	4.207355	0	11.20735	7.405166	
8	1	4.207355	0	12.20735	9.105517	
9	2	4.546487	36.3719	49.91838	19.046	
10	3	0.7056	0	10.7056	18.91704	15.79829
11	4	-3.78401	0	7.215988	18.25094	15.99915
12	3	0.7056	0	12.7056	18.55059	12.97788
13	2	4.546487	0	17.54649	19.61841	14.36196
14	1	4.207355	33.65884	51.86619	20.00797	19.52699
15	1	4.207355	0	19.20735	21.70832	20.31268
16	2	4.546487	0	20.54649	24.37442	21.31268
17	3	0.7056	0	17.7056	25.37442	21.96251



Activity 4: Let's create MA3, MA7 from the temperature data below.

- Ref: <https://www.kaggle.com/datasets/aakashjoshi123/tempsense-temperature-and-sensor-data>

	A	B	C	D	E	F	G	H	I
1	Temperatu	Sensor1	Sensor2	Sensor3	Sensor4	Sensor5	temp	MA3	MA7
2	25.43149	0.677916	0.208399	0.033098	0.145758	0.291933	25.43149		
3	24.48606	0.801851	0.445989	0.869082	0.499015	0.53149	24.48606		
4	20.05509	0.781817	0.040596	0.205773	0.517225	0.598457	20.05509	23.32421	
5	22.78169	0.65103	0.316018	0.945797	0.433242	0.378673	22.78169	22.44094	
6	25.29283	0.292357	0.893463	0.289188	0.490347	0.012502	25.29283	22.70987	
7	24.99036	0.943349	0.063948	0.718288	0.839039	0.983175	24.99036	24.35496	
8	23.98467	0.674063	0.194182	0.03629	0.041649	0.412192	23.98467	24.75595	23.86031
9	20.00053	0.248843	0.593327	0.43412	0.3016	0.021741	20.00053	22.99185	23.08446
10	30.06108	0.409016	0.671308	0.93226	0.170215	0.900515	30.06108	24.6821	23.88089
11	21.6801	0.93686	0.735942	0.226905	0.999312	0.693274	21.6801	23.91391	24.11304
12	29.049	0.450991	0.740448	0.698462	0.187613	0.586871	29.049	26.93006	25.00837
13	28.32323	0.236092	0.022577	0.374476	0.255245	0.766169	28.32323	26.35078	25.44128
14	25.54785	0.030743	0.59283	0.710625	0.425575	0.722542	25.54785	27.64003	25.52092
15	24.31045	0.04702	0.952721	0.815409	0.410519	0.488743	24.31045	26.06051	25.56746
16	22.59364	0.632929	0.002732	0.327118	0.106443	0.107972	22.59364	24.15065	25.93791
17	20.99256	0.45679	0.989962	0.285559	0.97552	0.683311	20.99256	22.63222	24.64241
18	26.23403	0.701553	0.560333	0.279732	0.894963	0.13436	26.23403	23.27341	25.29297
19	24.42264	0.201532	0.421957	0.875007	0.268946	0.860668	24.42264	23.88308	24.63206

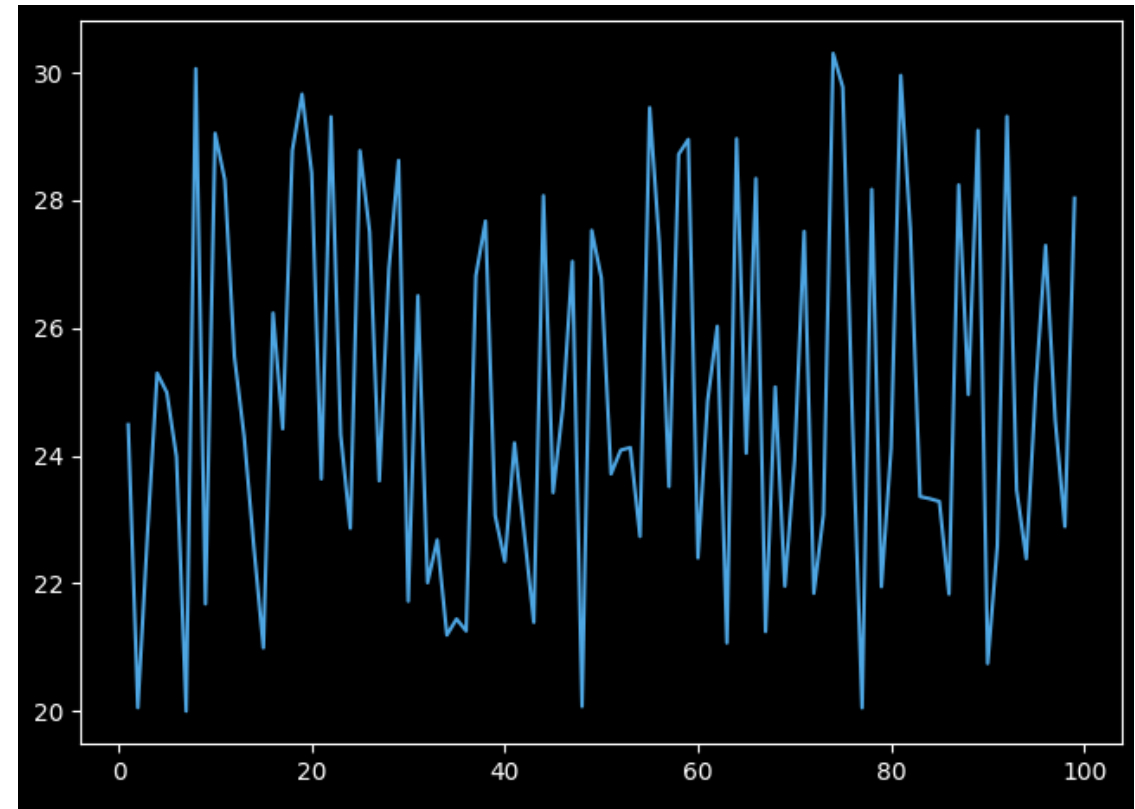


Activity 5: Let's create MA3, MA7 from the temperature data below with Python

```
import csv

import pandas as pd
df = pd.read_csv('Data.csv')
seldf = df["Temperature"][1:100]
print(seldf)

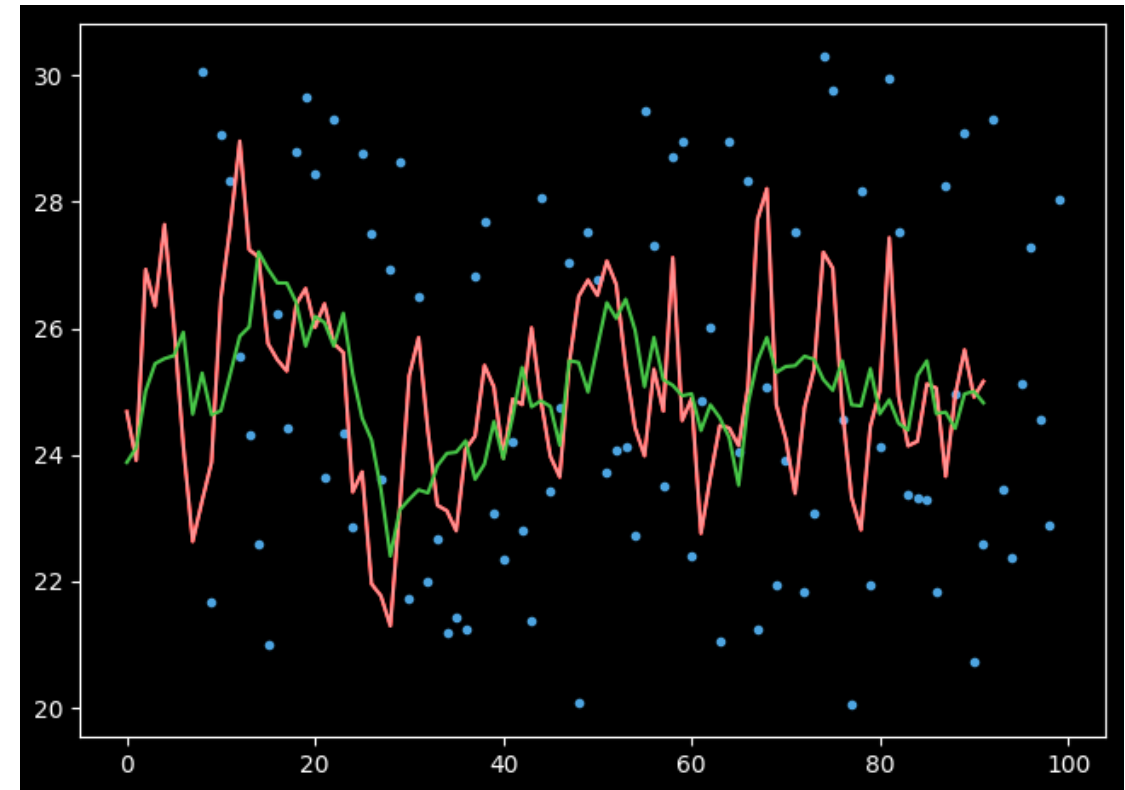
#Plot
import matplotlib.pyplot as plt
plt.clf() #Clear plot
plt.plot(seldf)
plt.show()
```



Activity 5: Let's create MA3, MA7 from the temperature data below with Python

```
#MA3 and MA7
ma3 = []
ma7 = []
for i in range(1,len(seldf)+1):
    print(seldf[i])
    if(i>=3):
        ma3.append(sum(seldf[i-3:i])/3)
    else:
        ma3.append(0)
    if(i>=7):
        ma7.append(sum(seldf[i-7:i])/7)
    else:
        ma7.append(0)

#Plot
import matplotlib.pyplot as mpl
mpl.clf() #Clear plot
mpl.plot(seldf, '.')
mpl.plot(ma3,color = 'r')
mpl.plot(ma7,color = 'g')
mpl.axis([7,90,20,30])
mpl.show()
```



Exponential Smoothing

- Exponential smoothing (ES) is a simple predictive method that utilizes the latest data to calculate the changing differences.

$$F_t = F_{t-1} + W (A_{t-1} - F_{t-1})$$

F : forecast value

A: Actual value

t: Time

W: Weight (between 0 to 1)

Example Exponential Smoothing

- The predicted car sales for February were 142, but the actual sales totalled 153, with an error of 0.2.

$$\begin{aligned}\text{Next month's forecasting} &= 142 + 0.2 \times (153 - 142) \\ &= 144.2\end{aligned}$$

Activity 12.6 Prediction with Exponential Smoothing

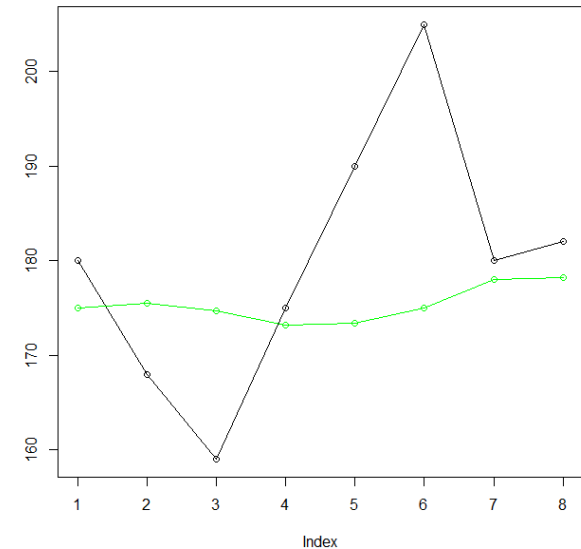
ไตรมาส	ยอดขายถ้าจริง (ต้น)	ค่าพยากรณ์ ($\alpha = 0.10$)
1	180	175
2	168	$176 = 175 + 0.1(180 - 175)$
3	159	$175 = 176 + 0.1(168 - 176)$
4	175	$173 = 175 + 0.1(159 - 175)$
5	190	173
6	205	175
7	180	178
8	182	178
9	?	179

```
#####  
##  
#ACTIVITY 12.5 Exponential Smoothing  
#####  
##
```

```
data = c(180,168,159,175,190,205,180,182)  
pred = c(175)  
weight = 0.1  
for (i in 1:(length(data)-1))  
{  
    pred[i+1] = pred[i]+weight*(data[i]-  
    pred[i])  
}  
print(pred)
```

```
MAD_err = sum(abs(data-pred))/length(data)  
print(MAD_err)
```

```
plot(data, type="o", ylab="Data")  
lines(pred,type="o", col="green")
```

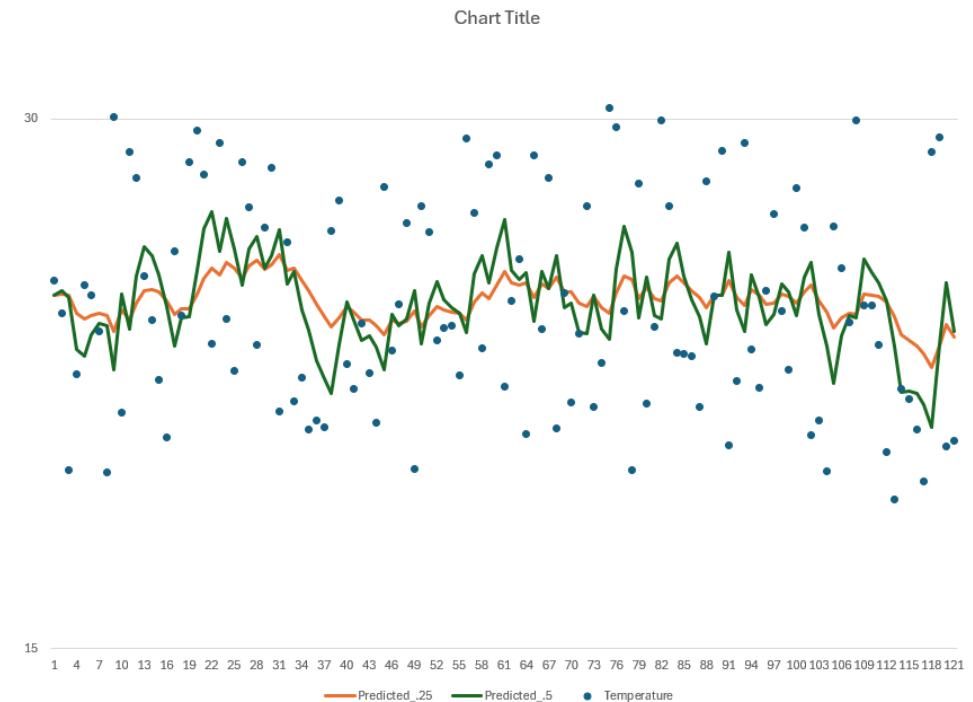


```
> pred  
[1] 175.0000 175.5000 174.7500 173.1750 173.3575 175.0218  
[7] 178.0196 178.2176  
>  
> MAD_err = sum(abs(data-pred))/length(data)  
> print(MAD_err)  
[1] 10.30732
```

Activity 6: Let's exponential smoothing from the temperature data below.

- Ref: <https://www.kaggle.com/datasets/aakashjoshi123/tempsense-temperature-and-sensor-data>

Weight	0.1	0.3
Temperature	Predicted_.25	Predicted_.5
25.43148902	25	25
24.48605608	25.0431489	25.12944671
20.05508693	24.98743962	24.93642952
22.78169014	24.49420435	23.47202674
25.29282629	24.32295293	23.26492576
24.99035529	24.41994027	23.87329592
23.98467188	24.47698177	24.20841373
20.00053149	24.42775078	24.14129118
30.06108448	23.98502885	22.89906327
21.68010448	24.59263441	25.04766963
29.04899829	24.30138142	24.03740009
28.32322763	24.77614311	25.54087955
25.54785012	25.13085156	26.37558397
24.31045181	25.17255142	26.12726382
22.59364391	25.08634145	25.58222021
20.99256279	24.8370717	24.68564732
26.2340349	24.45262081	23.57772196
24.42263795	24.63076222	24.37461584
28.78189406	24.60994979	24.38902247

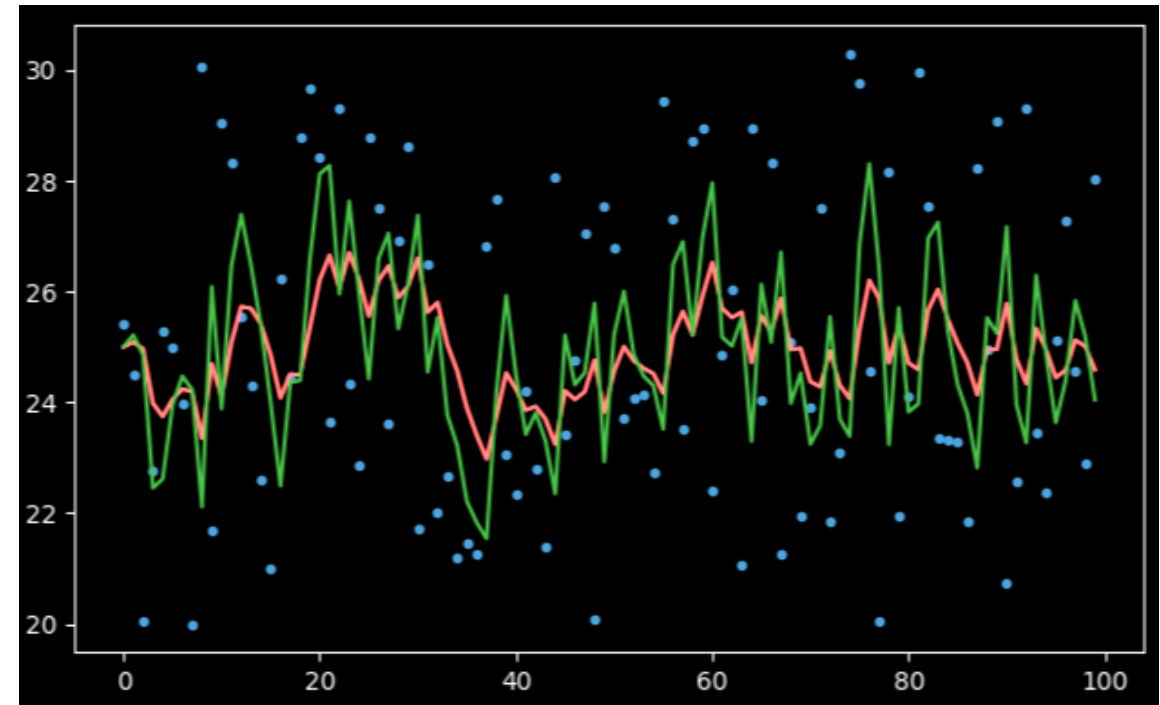


$$\text{pred}[i+1] = \text{pred}[i] + \text{weight} * (\text{data}[i] - \text{pred}[i])$$

Activity 7: Let's create exponential smoothing from the temperature data below with Python

```
#exponential smoothing
es2 = []
es5 = []
es2.append(25) #initial temperature
es5.append(25)
for i in range(1,len(seldf)):
    r = es2[i-1]+0.2*(seldf[i-1]-es2[i-1])
    es2.append(r)
    r = es5[i-1]+0.5*(seldf[i-1]-es5[i-1])
    es5.append(r)

#Plot
import matplotlib.pyplot as plt
plt.clf() #Clear plot
plt.plot(seldf, '.')
plt.plot(es2,color = 'r')
plt.plot(es5,color = 'g')
#mpl.axis([7,90,20,30])
plt.show() = df["Temperature"][1:100]
print(seldf)
```



4. Decomposition methods

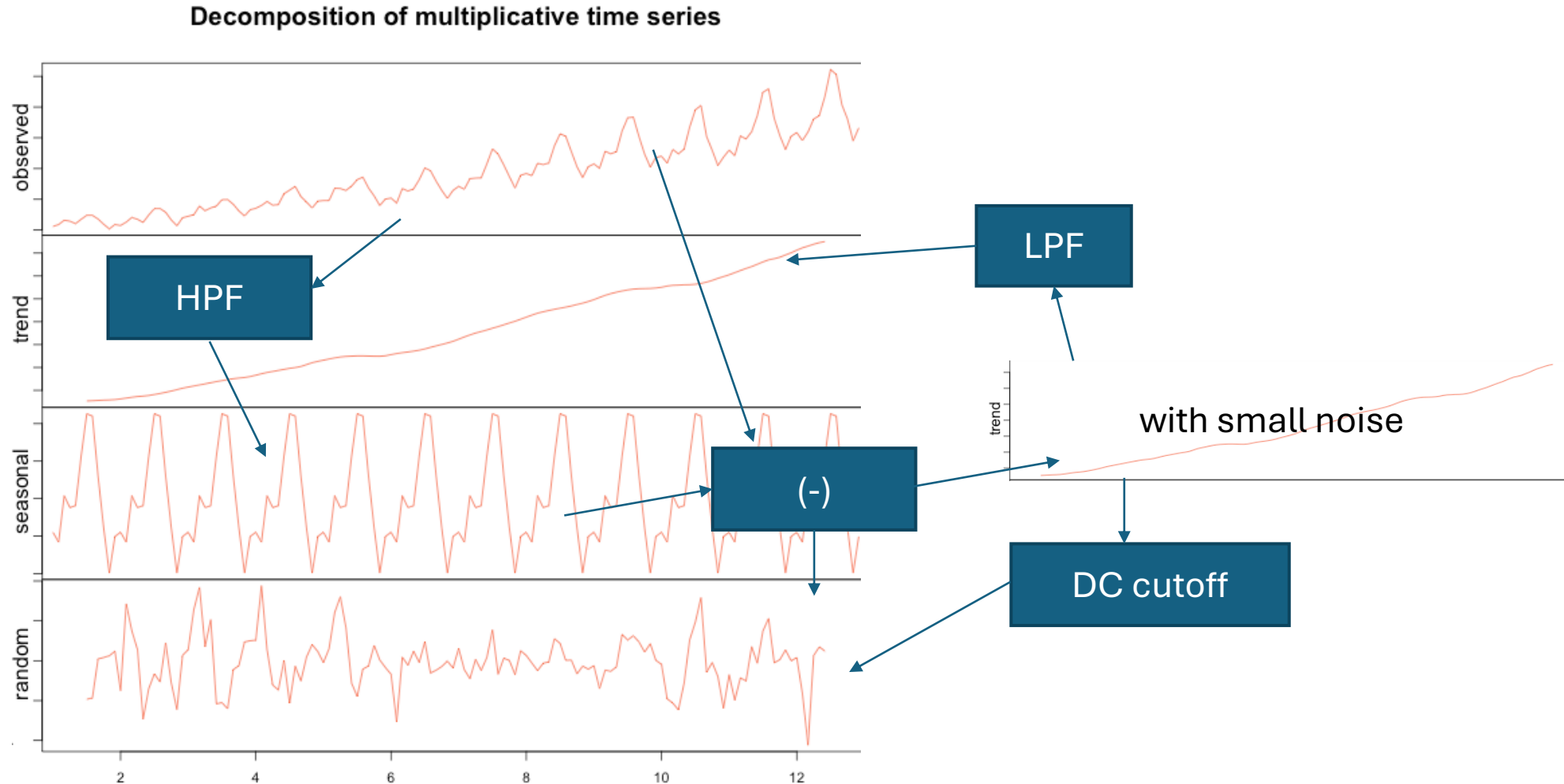
Decomposition

- Decomposition is separation a time series into components: trend, seasonality, cycle, and randomness.

$$X_t = f(T_t, C_t, S_t, R_t)$$

- Where;
 - t is time.
 - X is time series.
 - T is trend component.
 - C is cycle component.
 - S is seasonal component.
 - R is random component.

Decomposition process concept



Decomposition

The process of decomposition method:

1. For X_t , compute a moving average whose length N is equal to the length of seasonality.
2. Separate the N -period moving average in step 1 from the original data series to obtain trend and cyclicity.
3. Isolate the seasonal factors by averaging them for each of the periods making up the complete length of
4. seasonality.
5. Identify the appropriate form of the trend (linear or exponential) and calculate its value at each period T_t .
6. Separate the outcome of step 4 from step 2 to obtain the cyclical factor.
7. Separate the seasonality, trend, and cycle from the original data series to isolate the remaining randomness.

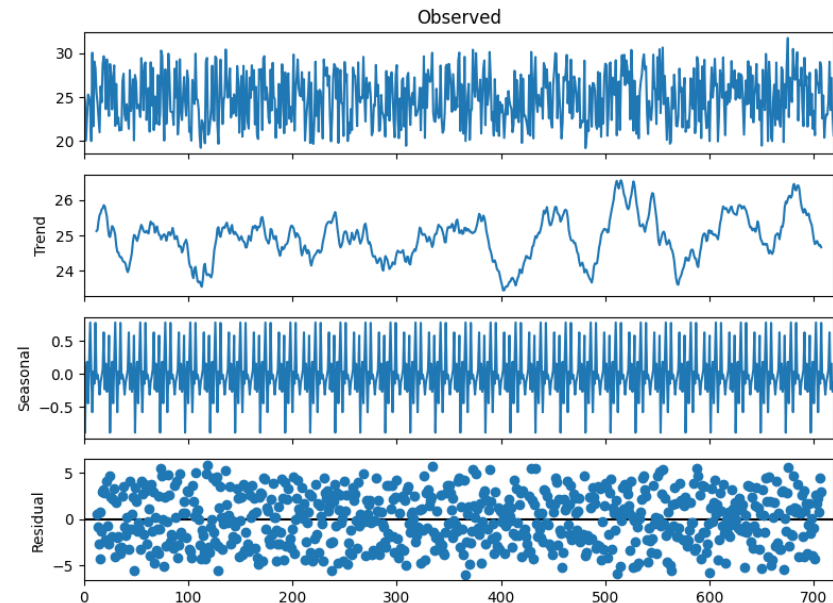
Activity 8: Decomposition from the temperature data with Python

```
import pandas as pd
df = pd.read_csv('Data.csv')
seldf = df["Temperature"][0:30*24]
print(seldf)

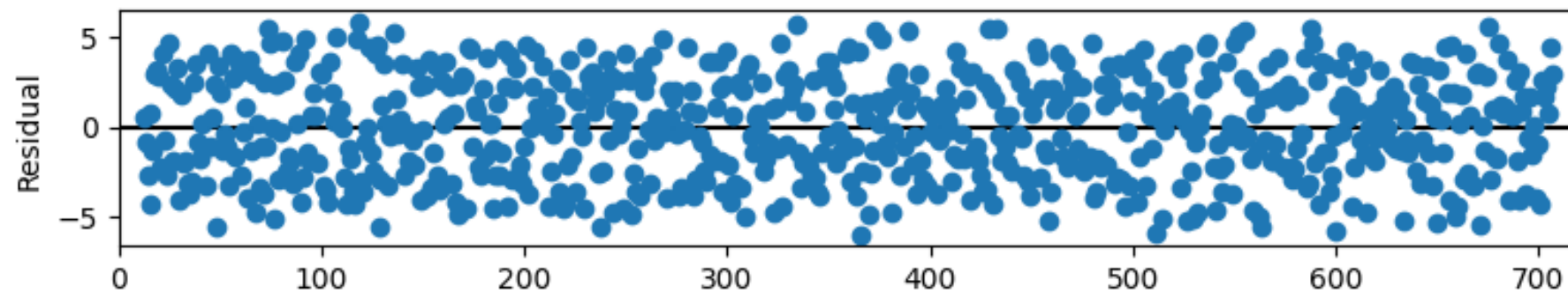
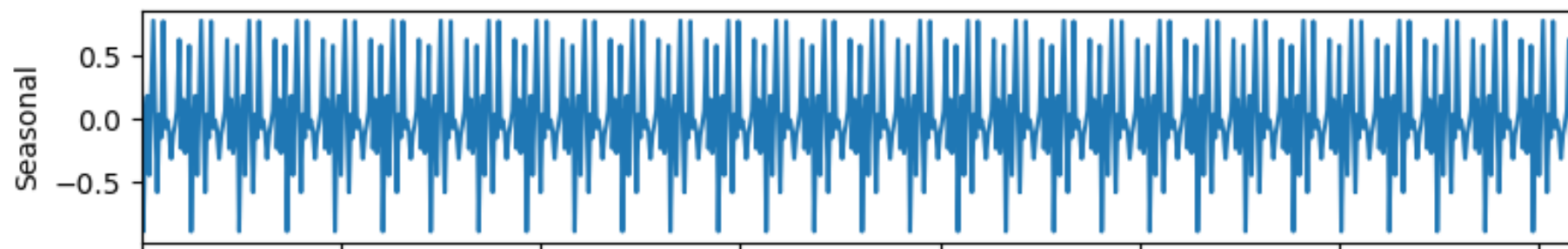
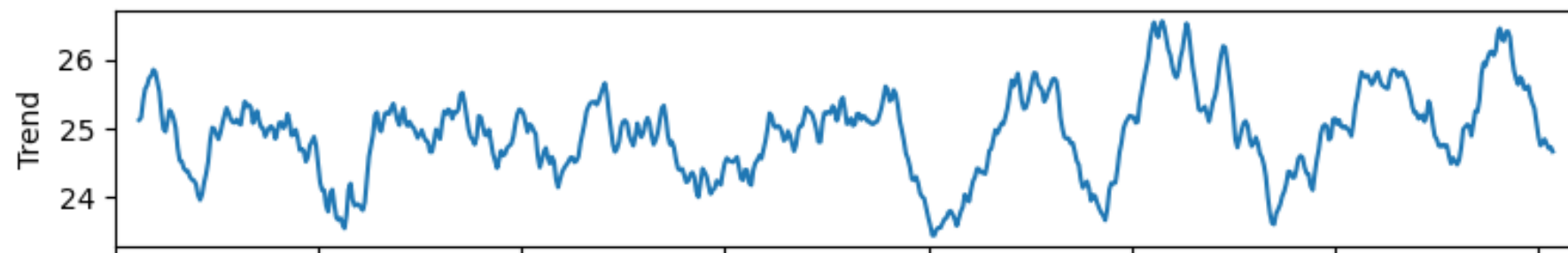
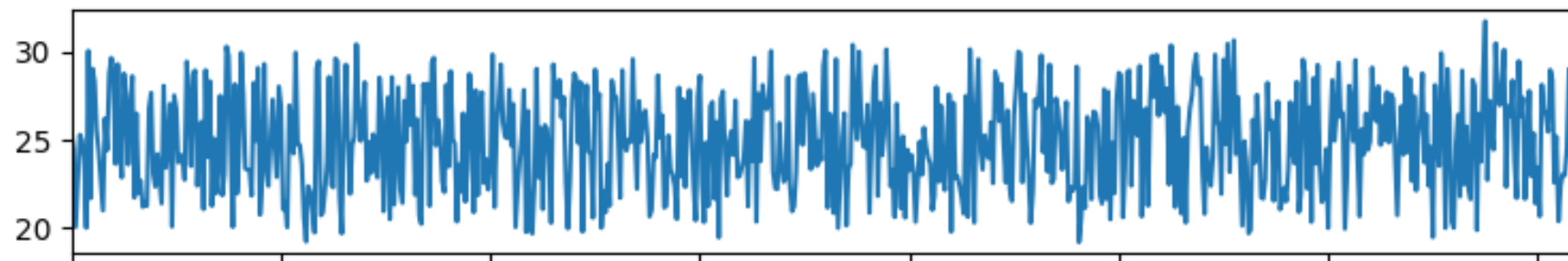
#PLOT ORIGINAL DATA
import matplotlib.pyplot as plt
plt.clf() #Clear plot
#plt.plot(seldf)
#plt.show()

#CREATE TIME SERIES
import numpy as np
np.random.seed(0)
date_rng = pd.date_range(start="2021-01-01",
periods=30*24, freq="D")
data = np.array(seldf) #Put obj pandas to
numpy
ts = pd.Series(data, index=date_rng)
print(ts)
```

```
#DECOMPOSITION
#pip install statsmodels
from statsmodels.tsa.seasonal import
seasonal_decompose
dec_result = seasonal_decompose(data,
model='additive',period=24)
dec_result.plot()
```



Observed



Quiz: Johnson & Johnson EPS

Johnson and Joneson had the earnings per share (EPS) from 1960 to 1980 as shown below. Let's analyze this data using a moving average.

0.71, 0.63, 0.85, 0.44, 0.61, 0.69, 0.92, 0.55, 0.72, 0.77, 0.92, 0.6, 0.83, 0.8, 1, 0.77, 0.92, 1, 1.24, 1, 1.16, 1.3, 1.45, 1.25, 1.26, 1.38, 1.86, 1.56, 1.53, 1.59, 1.83, 1.86, 1.53, 2.07, 2.34, 2.25, 2.16, 2.43, 2.7, 2.25, 2.79, 3.42, 3.69, 3.6, 3.6, 4.32, 4.32, 4.05, 4.86, 5.04, 5.04, 4.41, 5.58, 5.85, 6.57, 5.31, 6.03, 6.39, 6.93, 5.85, 6.93, 7.74, 7.83, 6.12, 7.74, 8.91, 8.28, 6.84, 9.54, 10.26, 9.54, 8.729999, 11.88, 12.06, 12.15, 8.91, 14.04, 12.96, 14.85, 9.99, 16.2, 14.67, 16.02, 11.61

Summary



Afternoon Section

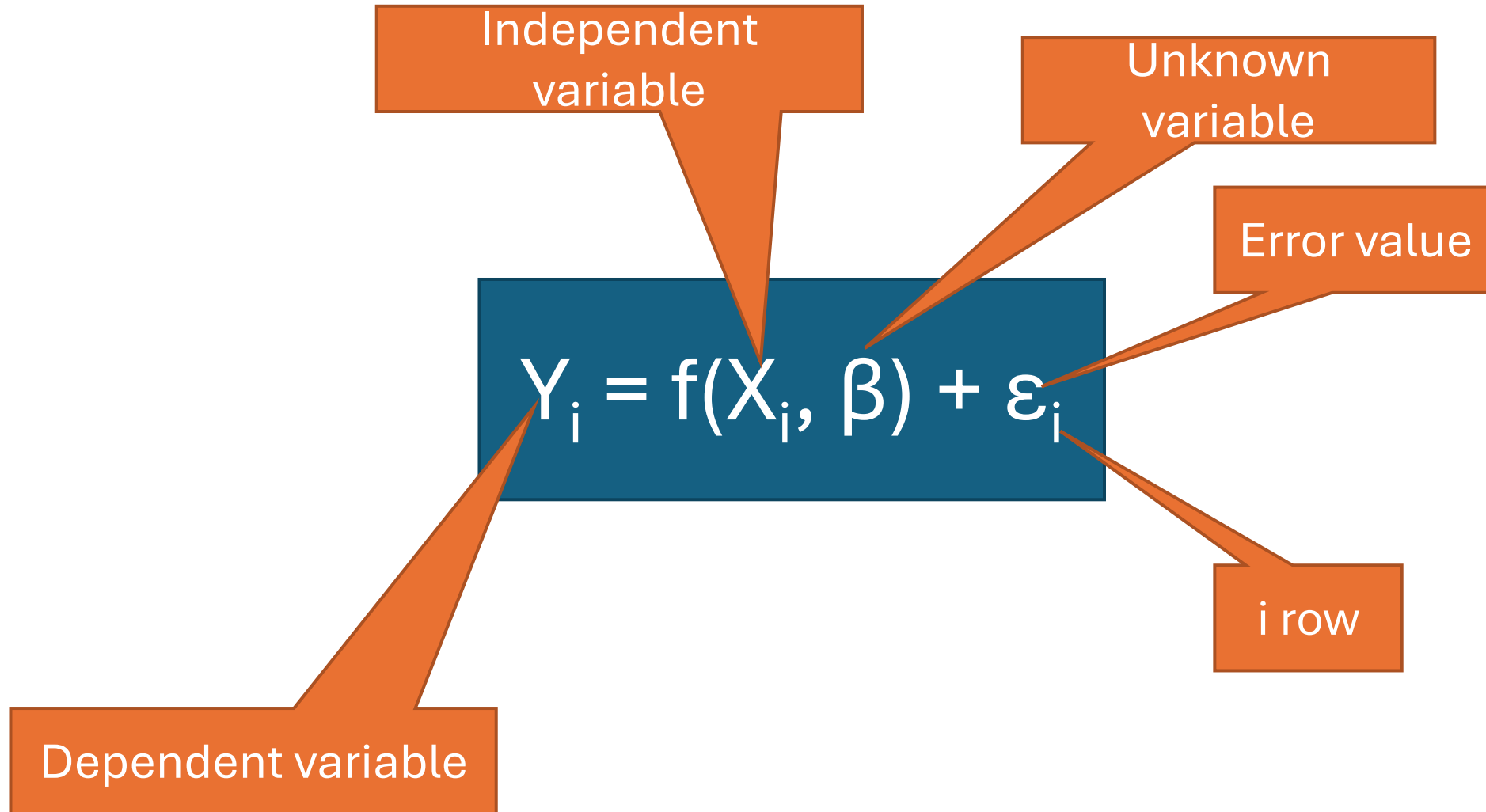
การติดตามสุขภาพและการวิเคราะห์แนวโน้ม

Topics

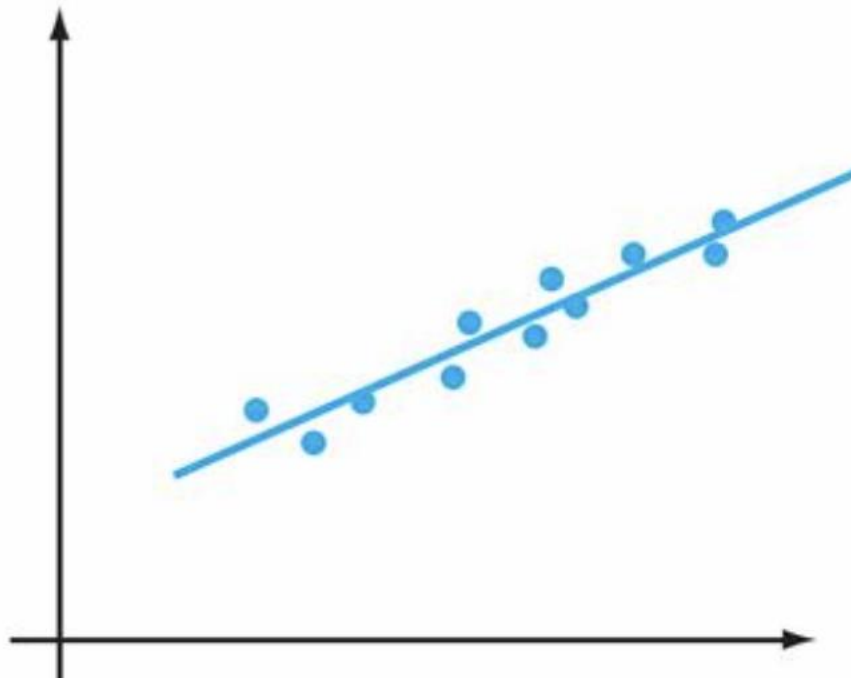
- Simple regression analysis with Excel/Python
 - การเก็บรวบรวมข้อมูลสุขภาพอย่างต่อเนื่อง (น้ำหนัก ความดัน ระดับน้ำตาล อัตราการเต้นของหัวใจ)
- Shiny dashboard
 - การวิเคราะห์แนวโน้มและความสัมพันธ์ระหว่างปัจจัยสุขภาพ ต่างๆ
 - การตรวจจับความผิดปกติและการแจ้งเตือน
 - การสร้างรายงานและแดชบอร์ดเพื่อติดตามสุขภาพ
 - **Workshop** : "ระบบติดตามสุขภาพผู้สูงอายุในชุมชน" ที่ รวบรวมข้อมูลสัญญาณชีพและกิจกรรมประจำวันของผู้สูงอายุ วิเคราะห์แนวโน้มเพื่อตรวจจับความเสี่ยงด้านสุขภาพล่วงหน้า และสร้างระบบแจ้งเตือนสำหรับผู้ดูแล
- Quiz

Simple Linear Regression

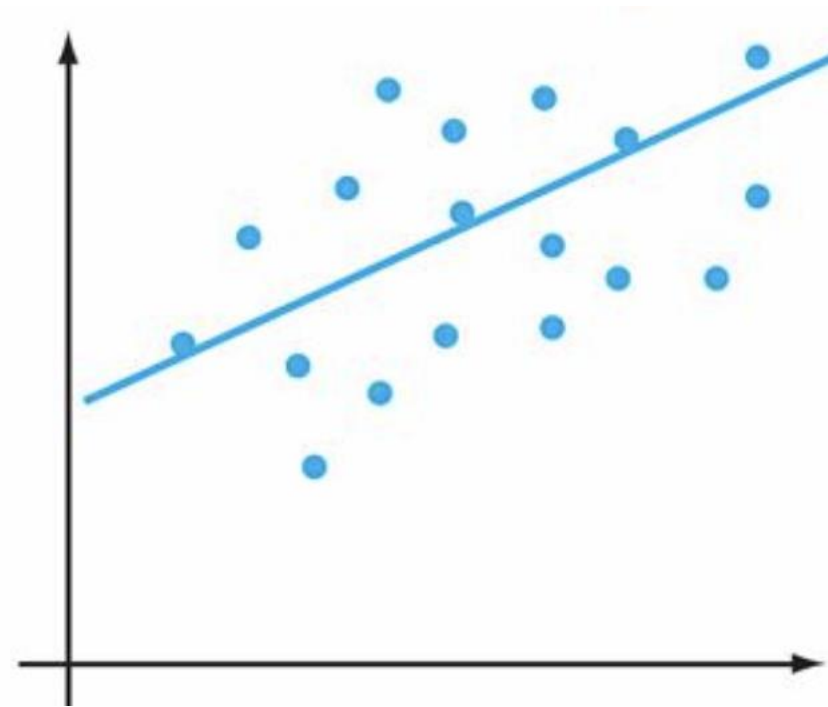
Variables in Regression



Let determine between A and B which one is the low r-square or residual.



(A)



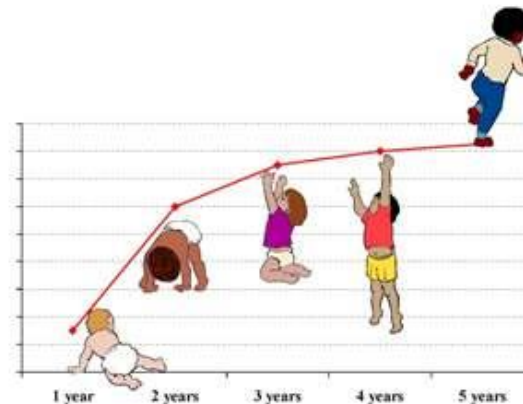
(B)

- Residuals is the distance between of y and $\hat{y}$.

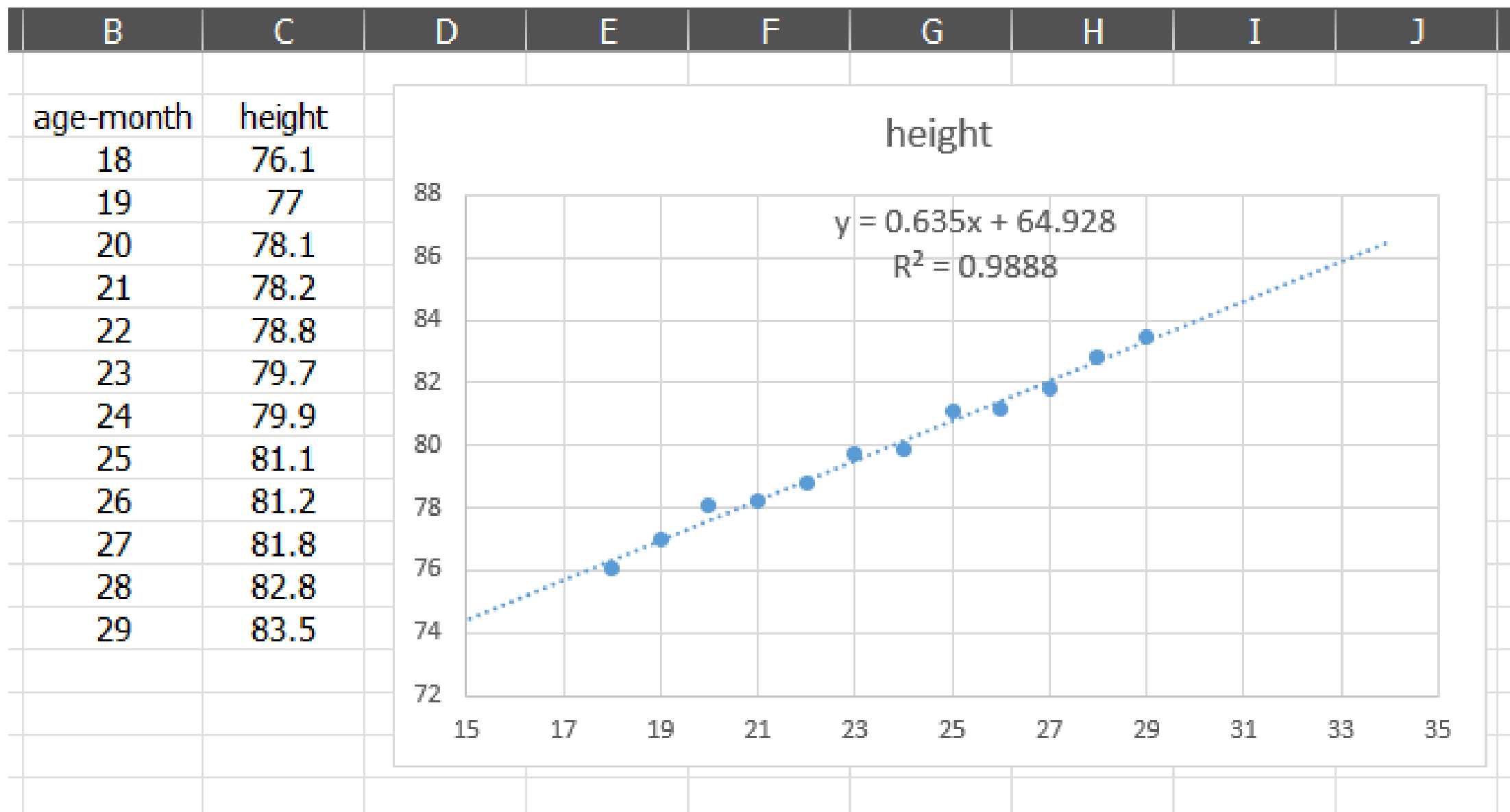
Activity 9 Baby height prediction with Excel

- A study collected the growth of the children and found an assumption that the age and height of the children are related. In this study, the researcher needs to display an equation for the growth of children.
- **Step1**: checking the data, relationship by time or variables.
- **Step2**: write the equation of **the simple linear regression**.

$$height = \beta_0 + \beta_1 age$$



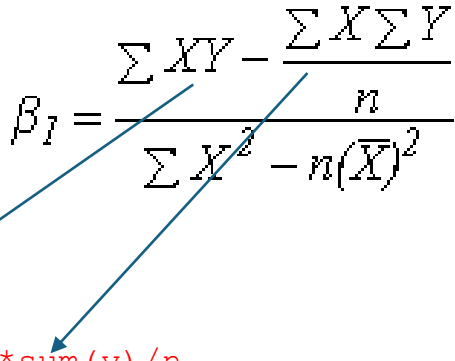
age	height
18	76.1
19	77
20	78.1
21	78.2
22	78.8
23	79.7
24	79.9
25	81.1
26	81.2
27	81.8
28	82.8
29	83.5



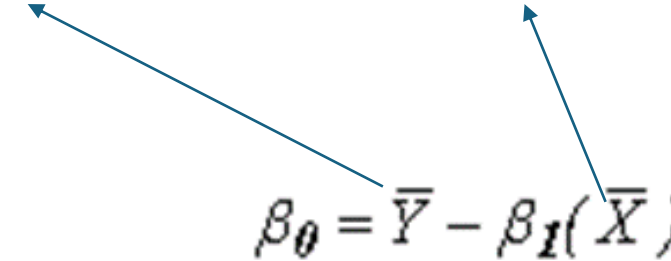
Activity10: SLR without library in Python

Step3 Calculate SLR

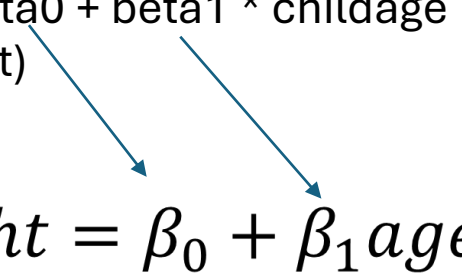
```
##PREPARING DATA
age = (18,19,20,21,22,23,24,25,26,27,28,29)
height = (76.1,77,78.1,78.2,78.8,79.7,
79.9,81.1,81.2,81.8,82.8,83.5)
#SIMPLE LINEAR REGRESSION
import numpy as np
import statistics as stat
x = np.array(age)
y = np.array(height)
n = len(age)
xy = x*y
xsquare = x**2
sp = sum(xy) - sum(x)*sum(y)/n
ss = sum(xsquare) - n * stat.mean(x)**2
```


$$\beta_1 = \frac{\sum XY - \frac{\sum X \sum Y}{n}}{\sum X^2 - n(\bar{X})^2}$$

```
beta1 = sp/ss
beta0 = stat.mean(y) - beta1 * stat.mean(x)
print(beta0)
print(beta1)
```


$$\beta_0 = \bar{Y} - \beta_1(\bar{X})$$

```
#PREDICTION
childage = 30
childheight = beta0 + beta1 * childage
print(childheight)
```


$$height = \beta_0 + \beta_1 age$$

```

10 #SIMPLE LINEAR REGRESSION
11 import numpy as np
12 import statistics as stat
13 x = np.array(age)
14 y = np.array(height)
15 n = len(age)
16 xy = x*y
17 xsquare = x**2
18 sp = sum(xy) - sum(x)*sum(y)/n
19 ss = sum(xsquare) - n * stat.mean(x)**2
20 beta1 = sp/ss
21 beta0 = stat.mean(y) - beta1 * stat.mean(x)
22 print(beta0)
23 print(beta1)
24
25
26 #PREDICTION
27 childage = 30
28 childeight = beta0 + beta1 * childage
29 print(childeight)

```

CONSOLE TERMINAL PROBLEMS OUTPUT PORTS DEBUG CONSOLE

Python 3.13.3 (MicrosoftStore) ▾

80.92582938388032

```

>>> childage = 30
... childeight = beta0 + beta1 * childage
... print(childeight)

```

81.35616113744086

>>>

>>>

>>>

>>>

>>> |

Python 3.13.3 (MicrosoftStore) ▾

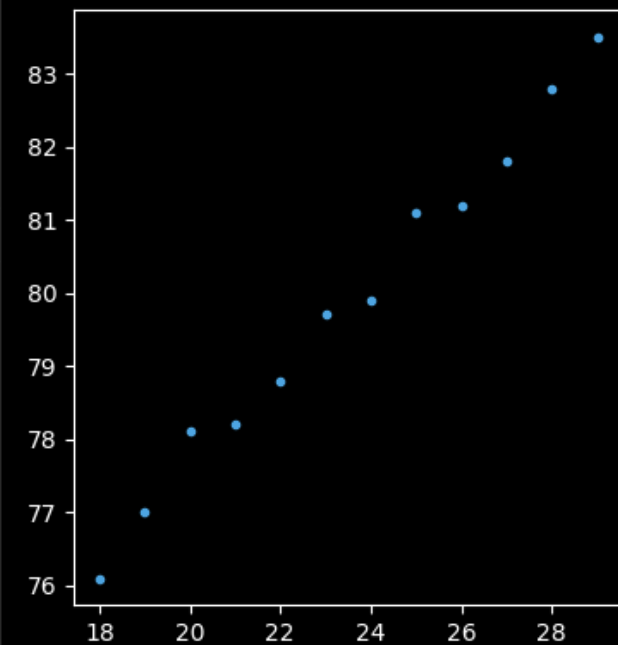
filter

VALUES

> age	(18, 19, 20, 21, 22, 23, 24, 25, 26, 27, ...
alpha	74.90118483412286
beta	0.2151658767772667
beta0	74.90118483412286
beta1	0.2151658767772667
childage	30
childeight	81.35616113744086
> height	(76.1, 77, 78.1, 78.2, 78.8, 79.7, 79.9, ...
n	12
sp	90.800000000000655
ss	422
> x	[18,19,20,21,22,23,24,25,26,27,28,29]
> xsquare	[324,361,400,441,484,529,576,625,67...

PLOTS

↩ ↪ | 📄 📄 📄 Fit ▾ 📄 Auto ▾ 📄 ▾ 🌙 ▾



Activity 10: SLR with Library in Python (1)

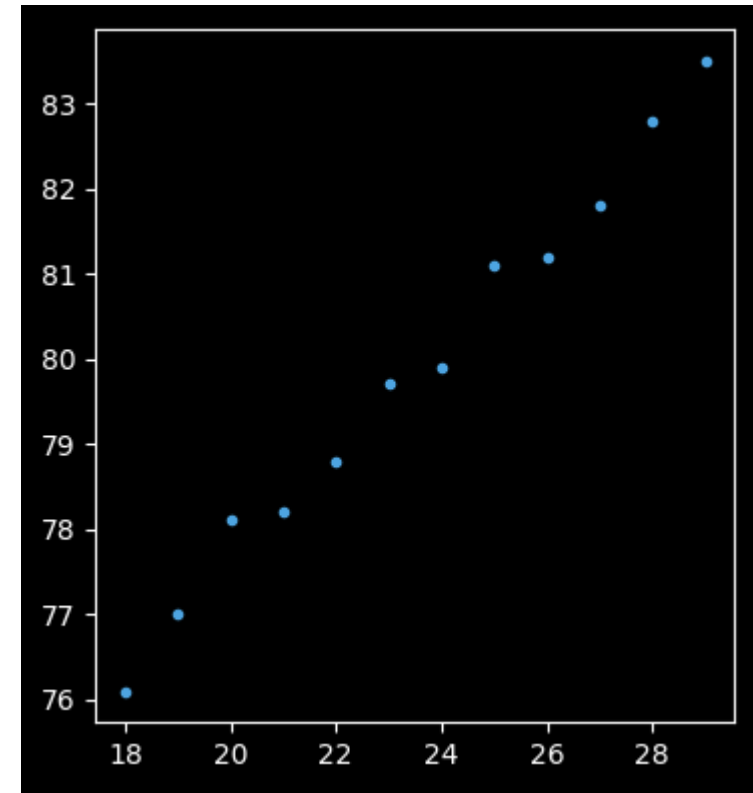
```
age = (18,19,20,21,22,23,24,25,26,27,28,29)
height =
(76.1,77,78.1,78.2,78.8,79.7,79.9,81.1,81.2,81.8,82.8,83.5)
```

```
#PLOT ORIGINAL DATA
import matplotlib.pyplot as plt
plt.clf() #Clear plot
plt.plot(age,height, '.')
plt.show()
```

```
#Method1
import numpy as np
from sklearn import linear_model
```

```
X = np.array(age).reshape((-1,1))
y = np.array(height)
model = linear_model.LinearRegression()
model.fit(X,y)
model.predict([[30]])
```

```
... model = linear_model.LinearRegression()
... model.fit(X,y)
... model.predict([[30]])
array([83.97727273])
```



Activity 10: SLR with Library in Python (2)

```
age = (18,19,20,21,22,23,24,25,26,27,28,29)
height =
(76.1,77,78.1,78.2,78.8,79.7,79.9,81.1,81.2,81
.8,82.8,83.5)

#Method2
import pandas as pd
import statsmodels.formula.api as smf

df = pd.DataFrame({'age':age,'height':height})
model = smf.ols(formula='height ~ age',
                data=df).fit()

# model summary
model.summary()
```

OLS Regression Results						
Dep. Variable:	height	R-squared:	0.989			
Model:	OLS	Adj. R-squared:	0.988			
Method:	Least Squares	F-statistic:	880.0			
Date:	Sat, 03 May 2025	Prob (F-statistic):	4.43e-11			
Time:	22:16:54	Log-Likelihood:	0.41930			
No. Observations:	12	AIC:	3.161			
Df Residuals:	10	BIC:	4.131			
Df Model:	1					
Covariance Type: nonrobust						
	coef	std err	t	P> t	[0.025	0.975]
Intercept	64.9283	0.508	127.709	0.000	63.796	66.061
age	0.6350	0.021	29.665	0.000	0.587	0.683
Omnibus:	0.869	Durbin-Watson:	2.408			
Prob(Omnibus):	0.648	Jarque-Bera (JB):	0.769			
Skew:	0.458	Prob(JB):	0.681			
Kurtosis:	2.164	Cond. No.	164.			



Shiny Dashboard for Healthy



Install shiny dashboard

```
pip install --upgrade pip wheel
```

```
C:\Users\supak>pip install --upgrade pip wheel
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in c:\users\supak\appdata\local\packages\pythonsoftw
fra8p0\localcache\local-packages\python313\site-packages (25.1)
Collecting pip
  Downloading pip-25.1.1-py3-none-any.whl.metadata (3.6 kB)
Collecting wheel
  Using cached wheel-0.45.1-py3-none-any.whl.metadata (2.3 kB)
Downloading pip-25.1.1-py3-none-any.whl (1.8 MB)
  1.8/1.8 MB 16.1 MB/s eta 0:00:00
```

```
pip install shiny
```

```
pip install --upgrade shiny htmltools
```

Install required dependencies:

```
cd dashboard
```

```
pip install -r requirements.txt
```

- Open and edit the app file: `dashboard\app.py`
- Run the app with ``shiny run app.py`` .

Filter controls

age

10

100

- chestpain
- ☒ asymptomatic
 - ☒ nonanginal
 - ☒ nontypical
 - ☒ typical



Number of people
303

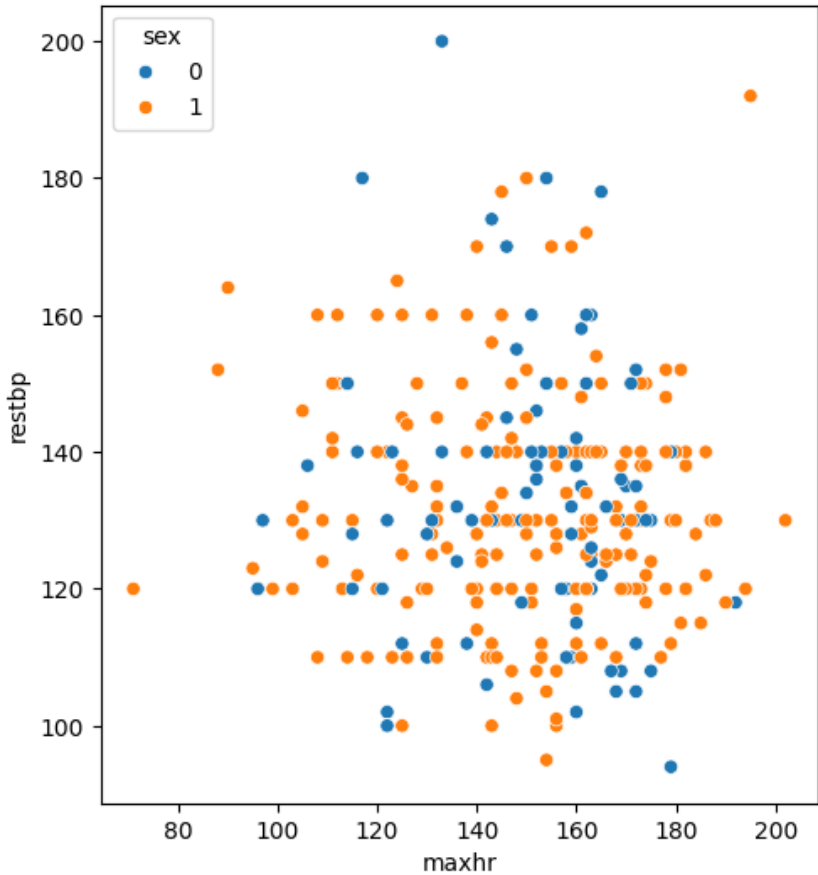


restbp
131.7



chol
246.7

Overall



Heart Data

chestpain	restbp	chol	maxhr	ahd
asymptomatic	160	286	108	Yes
asymptomatic	120	229	129	Yes
asymptomatic	140	268	160	Yes
asymptomatic	120	354	163	No
asymptomatic	130	254	147	Yes
asymptomatic	140	203	155	Yes
asymptomatic	140	192	148	No
asymptomatic	140	239	160	No
asymptomatic	130	206	132	Yes
asymptomatic	150	247	171	No
asymptomatic	110	167	114	Yes
asymptomatic	117	230	160	Yes
asymptomatic	135	234	161	No
asymptomatic	140	226	178	No
asymptomatic	120	177	120	Yes

Viewing rows 1 through 15 of 303

- <https://www.datacamp.com/tutorial/tutorial-for-recurrent-neural-network>
- How to Build a Real-time Network Traffic Dashboard with Python and Streamlit
 - <https://www.freecodecamp.org/news/build-a-real-time-network-traffic-dashboard-with-python-and-streamlit/>